

CURSO DE SQL Y BASES DE DATOS RELACIONALES

SQL

Inserción, Consulta y Administración de información en base de datos
relacionales basadas en SQL

Material de Referencia

miyoshikamium@gmail.mx

Versión 4.5 — 6 de julio de 2026

Resumen

Este documento constituye el material teórico de referencia para el Módulo 1 del *Curso de SQL y Bases de Datos Relacionales*. Se abordan los fundamentos históricos y conceptuales que dieron origen a los sistemas de gestión de bases de datos (SGBD) relacionales, la evolución del lenguaje SQL desde sus orígenes en los laboratorios de IBM hasta los estándares modernos, y los principios del modelo relacional propuesto por Edgar F. Codd. El objetivo es proveer al estudiante una base conceptual sólida que permita contextualizar cada construcción sintáctica de SQL dentro de un marco teórico coherente, facilitando un aprendizaje significativo y duradero.

Palabras clave: base de datos, modelo relacional, SQL, SGBD, álgebra relacional, normalización, ACID.

Índice

1. El Problema de los Datos: Una Perspectiva Histórica	4
1.1. Era pre-relacional (1950–1970)	4
2. Edgar F. Codd y el Modelo Relacional	4
2.1. Las 12 reglas de Codd	5
3. Nacimiento de SQL: De SEQUEL a SQL	5
3.1. El primer sistema comercial: Oracle	6
4. ¿Qué es una Base de Datos?	6
4.1. Analogía con el mundo real	6

5. Consulta de Datos: SELECT	7
5.1. Sintaxis completa y cláusulas	7
5.2. Tabla de ejemplo	8
5.3. Proyección de columnas: SELECT * y columnas específicas	9
5.4. Operadores aritméticos	10
5.5. Operadores de comparación	11
5.6. La cláusula WHERE: filtrado de filas	12
5.7. La cláusula ORDER BY: ordenamiento	12
5.8. La cláusula LIMIT y OFFSET: paginación	13
5.9. Expresiones con SELECT: funciones de cadena y fecha	13
5.10. Subconsultas en SELECT	14
5.11. Resumen visual de la estructura de SELECT	14
6. Consulta de Datos Relacionados	16
6.1. INNER JOIN	16
6.2. LEFT JOIN	17
6.3. RIGHT JOIN	18
6.4. UNION	19
7. Modificadores: Parte 1	21
7.1. DISTINCT	21
7.2. WHERE	22
7.3. ORDER BY	23
7.4. LIKE	24
7.5. AND, OR, NOT	24
7.6. LIMIT	26
8. Modificadores: Parte 2	27
8.1. Valores NULL	27
8.2. MIN y MAX	28
8.3. COUNT	28
8.4. SUM	29
8.5. AVG	29
8.6. IN	30
8.7. BETWEEN	31
8.8. ALIAS (AS)	31
8.9. CONCAT	32
8.10. GROUP BY	32
8.11. HAVING	33

8.12. CASE	34
8.13. IFNULL y COALESCE	35
8.14. Otros Modificadores	36
8.14.1. DISTINCT con funciones de agregación	36
8.14.2. ROLLUP: subtotales y total general	36
8.14.3. Expresiones aritméticas en GROUP BY y HAVING	36
8.14.4. FORMAT: formato de números en la salida	37
9. Almacenamiento de Datos Relacionados	38
9.1. Inserción en Tablas con Relación 1:1	38
9.2. Inserción en Tablas con Relación 1:N	39
9.3. Inserción en Tablas con Relación N:M	41
10.Escritura de Datos	44
10.1. INSERT	44
10.2. UPDATE	45
10.3. DELETE	46
11.Administración de la Base de Datos	49
11.1. CREATE DATABASE	49
11.2. DROP DATABASE	50
12.Conceptos Avanzados	51
12.1. INDEX	51
12.2. TRIGGER	52
12.3. VIEW	53
12.4. STORED PROCEDURE	54
12.5. Transacciones	55
12.6. Concurrencia	56
13.Conexión desde Código	59
13.1. Conectores y Drivers	59
13.2. SQL Injection	60
14.Otros Clientes SQL	62
15.PostgreSQL	62
16.Despliegue en la Nube	63
17.Próximos Pasos	65

1. El Problema de los Datos: Una Perspectiva Histórica

Desde los albores de la civilización, la humanidad ha necesitado almacenar y recuperar información de manera sistemática. Las tablillas de arcilla sumerias (3000 a.C.), los registros contables del Imperio Romano, o los libros de contabilidad de los comerciantes medievales son, en esencia, *sistemas de gestión de información*: colecciones de datos organizados con un propósito definido y un mecanismo de consulta.

La revolución informática de mediados del siglo XX transformó este problema milenario en una disciplina formal. Con la proliferación de computadoras en organizaciones bancarias, gubernamentales y empresariales durante los años 1950 y 1960, la necesidad de almacenar grandes volúmenes de datos de forma eficiente y recuperarlos con rapidez se volvió crítica.

1.1. Era pre-relacional (1950–1970)

Los primeros sistemas de gestión de datos surgieron como soluciones ad-hoc a problemas específicos. Predominaban dos modelos principales:

- **Modelo jerárquico:** Los datos se organizaban en una estructura de árbol. IBM desarrolló IMS (*Information Management System*) en 1966 para la misión Apolo de la NASA **date2003introduction**. Aunque eficiente para consultas predefinidas, era rígido: agregar una nueva relación entre datos requería reestructurar físicamente el sistema.
- **Modelo de red (CODASYL):** Propuesto por el comité CODASYL en 1969, permitía relaciones muchos-a-muchos mediante punteros físicos entre registros **codasyl1971**. Era más flexible que el jerárquico, pero extremadamente complejo de administrar y altamente dependiente de la estructura física del almacenamiento.

El problema fundamental de ambos modelos era la **dependencia físico-lógica**: los programas de aplicación debían conocer cómo estaban organizados físicamente los datos en disco para poder acceder a ellos. Un cambio en la estructura de almacenamiento implicaba reescribir todos los programas.

2. Edgar F. Codd y el Modelo Relacional

En junio de 1970, Edgar Frank Codd —matemático e investigador de IBM— publicó el artículo que cambiaría para siempre el mundo de las bases de datos: “*A Relational Model of Data for Large Shared Data Banks*” en la revista *Communications of the ACM* **codd1970**.

Definición: Modelo Relacional Codd

Un modelo de datos en el que la información se representa exclusivamente mediante **relaciones** (tablas bidimensionales), y en el que todas las operaciones

sobre los datos se expresan mediante un conjunto cerrado de operaciones del **álgebra relacional**, sin necesidad de conocer la organización física del almacenamiento.

La contribución de Codd fue revolucionaria por tres razones:

1. **Independencia de datos:** Los programas de aplicación operan sobre una representación lógica (tablas), completamente desacoplada de cómo los datos se almacenan físicamente.
2. **Fundamento matemático:** El modelo se basa en la teoría de conjuntos y la lógica de predicados de primer orden, lo que permite razonar formalmente sobre las operaciones.
3. **Lenguaje declarativo:** El programador especifica *qué* datos quiere, no *cómo* obtenerlos. El sistema optimiza el acceso automáticamente.

2.1. Las 12 reglas de Codd

En 1985, Codd formalizó su modelo en 12 reglas (numeradas del 0 al 12) que debe cumplir un sistema para ser considerado verdaderamente relacional **codd1985**. Las más relevantes para este curso son:

Regla	Descripción
0	Gestión mediante capacidades relacionales
1	Representación de la información como tablas
2	Acceso garantizado a cada dato (tabla + clave + columna)
3	Tratamiento sistemático de valores nulos
6	Actualización de vistas
9	Independencia de datos física y lógica
12	No subversión del modelo relacional

Tabla 1: Selección de las 12 reglas de Codd (1985).

3. Nacimiento de SQL: De SEQUEL a SQL

A raíz de la publicación de Codd, IBM inició el proyecto *System R* en 1973 en sus laboratorios de San José, California, con el objetivo de demostrar la viabilidad práctica del modelo relacional **chamberlin1974**. Donald D. Chamberlin y Raymond F. Boyce diseñaron un lenguaje de consulta denominado **SEQUEL** (*Structured English Query Language*).

El nombre fue posteriormente acortado a **SQL** (*Structured Query Language*) por razones legales de marca registrada. En paralelo, Michael Stonebraker y Eugene Wong desarrollaron **INGRES** en la Universidad de California, Berkeley (1973–1977), que introdujo su propio lenguaje de consulta llamado QUEL.

Nota importante

Aunque coloquialmente se pronuncia tanto “sequel” como “ese-cu-ele”, el estándar ISO/IEC lo denomina formalmente “SQL”. En este curso usaremos la pronunciación “ese-cu-ele” por su precisión técnica.

3.1. El primer sistema comercial: Oracle

En 1979, la empresa Relational Software Inc. (hoy Oracle Corporation), fundada por Larry Ellison, liberó el primer sistema de gestión de bases de datos relacionales comercial: **Oracle V2**. Paradójicamente, Oracle llegó al mercado antes que el propio IBM con su DB2 (1983), a pesar de que la tecnología subyacente provenía de los laboratorios de IBM.

4. ¿Qué es una Base de Datos?

Definición: Base de Datos

Una **base de datos** es una colección organizada, persistente y compartible de datos relacionados entre sí, diseñada para minimizar la redundancia y maximizar la integridad, de modo que múltiples aplicaciones y usuarios puedan acceder a ella de manera concurrente y controlada **date2003introduction**.

Esta definición merece descomponerse en sus atributos esenciales:

- **Organizada:** Los datos siguen una estructura formal (esquema) que define qué tipos de información se almacenan y cómo se relacionan entre sí.
- **Persistente:** Los datos sobreviven a la terminación de los procesos que los crean. A diferencia de las variables en memoria, los datos en una base de datos permanecen aunque el sistema se apague.
- **Compartible:** Múltiples usuarios o aplicaciones pueden acceder a los mismos datos simultáneamente, de forma controlada y sin interferencias.
- **Mínima redundancia:** El mismo dato no se repite innecesariamente en múltiples lugares, lo que reduce el riesgo de inconsistencias (un dato cambia en un lugar pero no en otro).

4.1. Analogía con el mundo real

Imaginemos el sistema de una biblioteca universitaria:

Ejemplo práctico

Una biblioteca almacena información sobre **libros**, **estudiantes** y **préstamos**. Sin una base de datos, cada ficha de préstamo copiaría el título completo del libro, el autor, el nombre del estudiante, su carrera, etc. Si un estudiante

cambia su dirección, habría que actualizar decenas de fichas físicas. Con una base de datos relacional, el estudiante aparece *una sola vez* en la tabla **Estudiantes**. Todos los préstamos simplemente hacen referencia a su identificador. Un cambio de dirección se realiza en un único lugar y se refleja automáticamente en todo el sistema.

5. Consulta de Datos: SELECT

La instrucción **SELECT** es, sin duda, la más utilizada y la más rica en posibilidades de todo el lenguaje SQL. Su propósito es **consultar** datos: extraer filas de una o varias tablas, aplicarles filtros, transformaciones y ordenamientos, y devolver un resultado tabular. A diferencia de **INSERT**, **UPDATE** o **DELETE**, **SELECT** es una operación de **sólo lectura**: nunca modifica los datos almacenados.

5.1. Sintaxis completa y cláusulas

La sintaxis formal de **SELECT** en MySQL/SQL estándar es la siguiente. Las partes entre corchetes [] son opcionales; las palabras en mayúsculas son palabras reservadas de SQL:

```
1 SELECT  [ALL | DISTINCT | DISTINCTROW]
2         lista_de_columnas | *
3 FROM    nombre_tabla  [[AS] alias_tabla]
4 [WHERE   condicion]
5 [GROUP BY columna [ASC | DESC], ...]
6 [HAVING  condicion_de_grupo]
7 [ORDER BY columna [ASC | DESC], ...]
8 [LIMIT   numero_de_filas [OFFSET desplazamiento]];
```

Listing 1: Sintaxis completa de SELECT.

Cada cláusula tiene un rol preciso dentro de la consulta. La [table 2](#) resume su función y el orden en que el SGBD las *evalúa* internamente (que no siempre coincide con el orden en que se escriben):

Cláusula	Orden eval.	Función
FROM	1	Determina la(s) tabla(s) fuente de datos.
WHERE	2	Filtra filas <i>antes</i> de agrupar.
GROUP BY	3	Agrupar filas con valores iguales en las columnas indicadas.
HAVING	4	Filtra grupos <i>después</i> de agrupar.
SELECT	5	Proyecta (selecciona) las columnas del resultado.
DISTINCT	6	Elimina filas duplicadas del resultado proyectado.
ORDER BY	7	Ordena el resultado final.
LIMIT	8	Recorta el número de filas devueltas.

Tabla 2: Cláusulas de SELECT y su orden de evaluación interno.

Nota importante

El orden de **escritura** de las cláusulas es fijo y debe respetarse exactamente como aparece en la sintaxis de arriba. Sin embargo, el SGBD las **evalúa** en un orden diferente (véase tabla 2). Esto explica, por ejemplo, por qué no se puede usar un alias definido en SELECT dentro de la cláusula WHERE: WHERE se evalúa *antes* de que exista el alias.

5.2. Tabla de ejemplo

Para ilustrar todos los ejemplos de esta sección usaremos una tabla llamada **empleados** con la siguiente estructura y datos:

```

1 CREATE TABLE empleados (
2     id            INT            PRIMARY KEY AUTO_INCREMENT,
3     nombre       VARCHAR(60)    NOT NULL,
4     departamento VARCHAR(40)    NOT NULL,
5     cargo        VARCHAR(40)    NOT NULL,
6     salario      DECIMAL(10,2)  NOT NULL,
7     fecha_ingreso DATE          NOT NULL
8 );
9
10 INSERT INTO empleados (nombre, departamento, cargo, salario
11     , fecha_ingreso) VALUES
12 ('Ana Garc a',      'Ventas',      'Gerente',      55000.00,
13     '2018-03-15'),
14 ('Luis Herrera',    'Ventas',      'Analista',     32000.00, '
15     2020-07-01'),
16 ('Sof a Ram rez',   'TI',          'Desarrolladora',
17     48000.00, '2019-11-20'),
18 ('Carlos Mendoza',  'TI',          'DBA',          51000.00, '
19     2017-05-10'),
20 ('Elena Torres',    'RRHH',        'Coordinadora', 38000.00,
21     '2021-02-28'),

```



```
16 ('Marco Villanueva', 'Ventas', 'Analista', 31000.00, '  
    2022-09-05'),  
17 ('Patricia Luna', 'TI', 'Desarrolladora',  
    47000.00, '2020-04-14'),  
18 ('Roberto Salas', 'RRHH', 'Gerente', 52000.00, '  
    2016-08-22'),  
19 ('Diana Castillo', 'Finanzas', 'Analista', 39000.00, '  
    2023-01-10'),  
20 ('Javier Moreno', 'Finanzas', 'Gerente', 58000.00, '  
    2015-06-30');
```

Listing 2: Creación e inicialización de la tabla empleados.

5.3. Proyección de columnas: SELECT * y columnas específicas

La **proyección** es la operación de elegir qué columnas aparecerán en el resultado. El asterisco `*` es un comodín que significa “todas las columnas”.

```
1 -- Devuelve todas las columnas de todos los empleados
2 SELECT *
3 FROM empleados;
```

Listing 3: Seleccionar todas las columnas.

```
1 -- S lo nombre, departamento y salario
2 SELECT nombre, departamento, salario
3 FROM empleados;
```

Listing 4: Seleccionar columnas específicas.

Nota importante

En producción se recomienda **evitar** `SELECT *` porque: (1) transfiere columnas innecesarias por la red, (2) si alguien agrega una columna sensible a la tabla, la consulta la expondrá automáticamente, (3) dificulta la legibilidad del código. Úsalo sólo para exploración interactiva.

También es posible incluir **expresiones** en la lista de columnas: literales, operaciones aritméticas o llamadas a funciones:

```
1 SELECT
2     nombre ,
3     salario ,
4     salario * 12          AS salario_anual ,
5     salario * 0.10       AS bono_estimado ,
6     'Activo'             AS estado    -- literal de texto
7 FROM empleados;
```

Listing 5: Columnas calculadas y literales en SELECT.

5.4. Operadores aritméticos

Dentro de `SELECT` (y también en `WHERE`) se pueden usar operadores aritméticos estándar sobre columnas numéricas:

Op.	Nombre	Ejemplo
+	Suma	<code>salario + 5000</code>
-	Resta	<code>salario - descuento</code>
*	Multiplicación	<code>salario * 1.15</code>
/	División	<code>salario / 14</code>
%	Módulo (resto)	<code>id % 2</code> (par/impar)

Tabla 3: Operadores aritméticos en SQL.

```

1 SELECT
2     nombre,
3     salario,
4     salario * 1.15      AS salario_con_aumento,
5     (salario / 30)      AS salario_diario,
6     salario - 5000      AS salario_base
7 FROM empleados
8 WHERE departamento = 'TI';

```

Listing 6: Uso de operadores aritméticos.

5.5. Operadores de comparación

Los operadores de comparación se usan principalmente en la cláusula `WHERE` para filtrar filas. Devuelven un valor booleano (`TRUE` o `FALSE`):

Operador	Significado	Ejemplo
<code>=</code>	Igual a	<code>departamento = 'Ventas'</code>
<code><></code> o <code>!=</code>	Distinto de	<code>cargo <> 'Gerente'</code>
<code>></code>	Mayor que	<code>salario > 40000</code>
<code><</code>	Menor que	<code>salario < 35000</code>
<code>>=</code>	Mayor o igual que	<code>salario >= 50000</code>
<code><=</code>	Menor o igual que	<code>salario <= 48000</code>

Tabla 4: Operadores de comparación en SQL.

```

1  -- Empleados con salario mayor a 45,000
2  SELECT nombre, cargo, salario
3  FROM empleados
4  WHERE salario > 45000;
5
6  -- Empleados que NO son del departamento de TI
7  SELECT nombre, departamento
8  FROM empleados
9  WHERE departamento <> 'TI';
10
11 -- Empleados con salario entre 30,000 y 40,000 (inclusive)
12 SELECT nombre, salario
13 FROM empleados
14 WHERE salario >= 30000
15     AND salario <= 40000;

```

Listing 7: Filtros con operadores de comparación.

Ejemplo práctico

Una analogía útil: la cláusula `WHERE` funciona exactamente como el filtro de una hoja de cálculo. Si tienes una columna `salario` y aplicas el filtro “mayor que 45,000”, sólo se muestran las filas que cumplen esa condición. SQL hace exactamente lo mismo, pero expresado como texto.

5.6. La cláusula `WHERE`: filtrado de filas

`WHERE` permite especificar una **condición de filtro** que cada fila debe satisfacer para aparecer en el resultado. Sólo las filas para las que la condición evalúa a `TRUE` son incluidas; las que evalúan a `FALSE` o `NULL` son descartadas.

```
1  -- Todos los empleados del departamento de Finanzas
2  SELECT *
3  FROM    empleados
4  WHERE   departamento = 'Finanzas';
5
6  -- Gerentes con salario mayor o igual a 50,000
7  SELECT nombre, cargo, salario
8  FROM    empleados
9  WHERE   cargo      = 'Gerente'
10     AND  salario >= 50000;
```

Listing 8: Uso básico de `WHERE`.

5.7. La cláusula `ORDER BY`: ordenamiento

`ORDER BY` ordena el resultado final por una o más columnas. El orden por defecto es **ascendente** (`ASC`); se puede invertir con **descendente** (`DESC`).

```
1  -- Ordenar por salario de mayor a menor
2  SELECT nombre, departamento, salario
3  FROM    empleados
4  ORDER BY salario DESC;
5
6  -- Ordenar primero por departamento ( A Z )
7  -- y dentro de cada departamento por salario ( m a y o r m e n o r )
8  SELECT nombre, departamento, salario
9  FROM    empleados
10 ORDER BY departamento ASC,
11          salario DESC;
```

Listing 9: Ordenamiento simple y compuesto.

Nota importante

Sin `ORDER BY`, el SGBD **no garantiza** ningún orden particular en los resultados. El orden puede cambiar entre ejecuciones, entre versiones del SGBD o según

los índices existentes. Si el orden importa, siempre especifícalo explícitamente.

5.8. La cláusula LIMIT y OFFSET: paginación

LIMIT restringe el número de filas devueltas. OFFSET indica cuántas filas saltarse antes de comenzar a devolver. Juntos permiten implementar **paginación**.

```

1  -- Los 3 empleados con mayor salario
2  SELECT nombre, salario
3  FROM    empleados
4  ORDER BY salario DESC
5  LIMIT 3;
6
7  -- P gina 2 (filas 4 a 6) de una lista de 3 por p gina
8  SELECT nombre, salario
9  FROM    empleados
10 ORDER BY salario DESC
11 LIMIT 3 OFFSET 3;
12
13 -- Sintaxis alternativa equivalente (MySQL): LIMIT offset,
14    count
15 SELECT nombre, salario
16 FROM    empleados
17 ORDER BY salario DESC
18 LIMIT 3, 3;    -- salta 3, devuelve 3

```

Listing 10: LIMIT y OFFSET para paginación.

5.9. Expresiones con SELECT: funciones de cadena y fecha

SELECT puede invocar funciones integradas del SGBD sobre las columnas. Las más utilizadas en MySQL son:

Función	Descripción
UPPER(s) / LOWER(s)	Convierte a mayúsculas / minúsculas.
LENGTH(s)	Longitud en bytes de la cadena.
SUBSTRING(s, inicio, n)	Extrae <i>n</i> caracteres desde <i>inicio</i> .
TRIM(s)	Elimina espacios al inicio y al final.
CONCAT(s1, s2, ...)	Concatena cadenas.
NOW()	Fecha y hora actuales.
YEAR(fecha)	Extrae el año de una fecha.
DATEDIFF(f1, f2)	Días entre dos fechas.
ROUND(n, decimales)	Redondea un número.

Tabla 5: Funciones integradas frecuentes en SELECT.

```

1 SELECT
2     UPPER(nombre)                AS
3     nombre_mayusculas ,
4     LENGTH(nombre)              AS longitud_nombre
5     ,
6     YEAR(fecha_ingreso)         AS anio_ingreso ,
7     DATEDIFF(NOW(), fecha_ingreso) / 365 AS
8     anios_en_empresa ,
9     ROUND(salario / 12, 2)      AS salario_mensual
10 FROM empleados;

```

Listing 11: Uso de funciones de cadena, fecha y número.

5.10. Subconsultas en SELECT

Una **subconsulta** (o *subquery*) es un SELECT anidado dentro de otro. En la lista de columnas se puede incluir una subconsulta *escalar* (que devuelve exactamente un valor):

```

1 -- Para cada empleado, muestra cu nto gana respecto
2 -- al salario promedio de toda la empresa
3 SELECT
4     nombre ,
5     salario ,
6     (SELECT AVG(salario) FROM empleados) AS
7     promedio_empresa ,
8     salario - (SELECT AVG(salario) FROM empleados) AS
9     diferencia
10 FROM empleados
11 ORDER BY diferencia DESC;

```

Listing 12: Subconsulta escalar en la lista de columnas.

5.11. Resumen visual de la estructura de SELECT

A modo de síntesis, la siguiente figura representa gráficamente cómo fluyen los datos a través de cada cláusula hasta producir el resultado final:

Ejemplo práctico

Flujo de evaluación de una consulta SELECT

1. FROM — Se carga(n) la(s) tabla(s) fuente en memoria de trabajo.
2. WHERE — Se descartan las filas que no cumplen la condición.
3. GROUP BY — Las filas restantes se agrupan por los valores de las columnas indicadas.
4. HAVING — Se descartan los grupos que no cumplen la condición de grupo.
5. SELECT — De cada grupo (o fila si no hay GROUP BY) se calculan y proyectan

las columnas indicadas.

6. **DISTINCT** — Se eliminan filas duplicadas del resultado proyectado.
7. **ORDER BY** — El resultado se ordena.
8. **LIMIT / OFFSET** — Se recorta la cantidad de filas devueltas.

```
1  -- "Los 3 departamentos donde el salario promedio
2  -- supera los 40,000, ordenados de mayor a menor promedio,
3  -- excluyendo el departamento de RRHH."
4  SELECT
5      departamento,
6      COUNT(*)           AS num_empleados,
7      AVG(salario)       AS salario_promedio,
8      MAX(salario)       AS salario_maximo
9  FROM      empleados
10 WHERE     departamento <> 'RRHH'           -- filtra filas
11 GROUP BY  departamento                   -- agrupa
12 HAVING    AVG(salario) > 40000           -- filtra grupos
13 ORDER BY  salario_promedio DESC         -- ordena
14 LIMIT     3;                           -- recorta
```

Listing 13: Consulta que usa todas las cláusulas.

6. Consulta de Datos Relacionados

Las consultas relacionales permiten combinar información distribuida en múltiples tablas mediante operaciones de *join* y unión. SQL ofrece distintos tipos de combinaciones según el comportamiento deseado ante filas sin coincidencia.

Nota importante

Para los ejemplos de esta sección se utilizan las tablas `clientes` y `pedidos`, relacionadas por la columna `cliente_id`.

Tabla 6: Tabla clientes

cliente_id	nombre
1	Ana García
2	Luis Pérez
3	María López
4	Carlos Ruiz

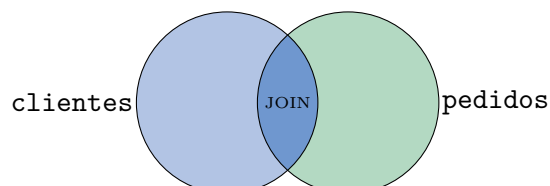
Tabla 7: Tabla pedidos

pedido_id	cliente_id	monto
101	1	350.00
102	2	120.50
103	2	89.00
104	5	200.00

6.1. INNER JOIN

Definición: INNER JOIN

Devuelve únicamente las filas que tienen **coincidencia en ambas tablas**. Las filas sin par en la tabla opuesta quedan excluidas del resultado.



Sintaxis general:

```
1 SELECT columnas
2 FROM tabla_izquierda
3 INNER JOIN tabla_derecha
4     ON tabla_izquierda.clave = tabla_derecha.clave;
```


Listing 14: Sintaxis de INNER JOIN

Ejemplo práctico

Obtener los pedidos junto con el nombre del cliente que los realizó:

```

1 SELECT c.nombre,
2       p.pedido_id,
3       p.monto
4 FROM clientes AS c
5 INNER JOIN pedidos AS p
6     ON c.cliente_id = p.cliente_id;

```

Listing 15: INNER JOIN entre clientes y pedidos

Resultado:

nombre	pedido_id	monto
Ana García	101	350.00
Luis Pérez	102	120.50
Luis Pérez	103	89.00

Los clientes María López y Carlos Ruiz no aparecen porque no tienen pedidos. El pedido 104 tampoco aparece porque su cliente_id = 5 no existe en clientes.

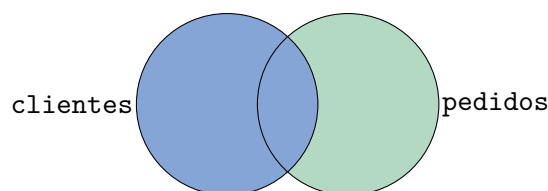
Nota importante

La palabra clave INNER es opcional; JOIN a secas equivale a INNER JOIN en todos los motores SQL principales.

6.2. LEFT JOIN

Definición: LEFT JOIN

Devuelve **todas las filas de la tabla izquierda** más las coincidencias de la tabla derecha. Cuando no existe par, las columnas de la tabla derecha se rellenan con NULL.



Sintaxis general:

```

1 SELECT columnas
2 FROM tabla_izquierda

```

```

3 LEFT JOIN tabla_derecha
4   ON tabla_izquierda.clave = tabla_derecha.clave;

```

Listing 16: Sintaxis de LEFT JOIN

Ejemplo práctico

Listar todos los clientes y, si tienen pedidos, mostrar su monto:

```

1 SELECT c.nombre,
2        p.pedido_id,
3        p.monto
4 FROM clientes AS c
5 LEFT JOIN pedidos AS p
6   ON c.cliente_id = p.cliente_id;

```

Listing 17: LEFT JOIN: todos los clientes con sus pedidos

Resultado:

nombre	pedido_id	monto
Ana García	101	350.00
Luis Pérez	102	120.50
Luis Pérez	103	89.00
María López	NULL	NULL
Carlos Ruiz	NULL	NULL

Los clientes sin pedidos aparecen con NULL en las columnas de pedidos.

Caso de uso habitual — detectar huérfanos:

```

1 SELECT c.nombre
2 FROM clientes AS c
3 LEFT JOIN pedidos AS p
4   ON c.cliente_id = p.cliente_id
5 WHERE p.pedido_id IS NULL;

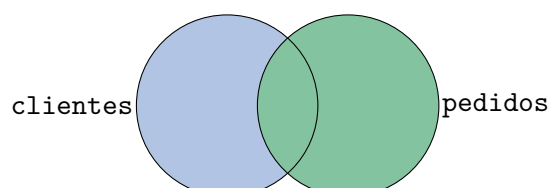
```

Listing 18: Clientes que nunca han realizado un pedido

6.3. RIGHT JOIN

Definición: RIGHT JOIN

Devuelve **todas las filas de la tabla derecha** más las coincidencias de la tabla izquierda. Las columnas de la tabla izquierda se rellenan con NULL cuando no existe par. Es el simétrico de LEFT JOIN.



Sintaxis general:

```

1 SELECT columnas
2 FROM tabla_izquierda
3 RIGHT JOIN tabla_derecha
4     ON tabla_izquierda.clave = tabla_derecha.clave;

```

Listing 19: Sintaxis de RIGHT JOIN

Ejemplo práctico

Mostrar todos los pedidos e incluir el nombre del cliente si existe:

```

1 SELECT c.nombre,
2         p.pedido_id,
3         p.monto
4 FROM clientes AS c
5 RIGHT JOIN pedidos AS p
6     ON c.cliente_id = p.cliente_id;

```

Listing 20: RIGHT JOIN: todos los pedidos con datos del cliente

Resultado:

nombre	pedido_id	monto
Ana García	101	350.00
Luis Pérez	102	120.50
Luis Pérez	103	89.00
NULL	104	200.00

El pedido 104 aparece con NULL en nombre porque su cliente_id = 5 no existe en la tabla clientes.

Nota importante

En la práctica, RIGHT JOIN se usa con menos frecuencia que LEFT JOIN. Cualquier consulta con RIGHT JOIN puede reescribirse como un LEFT JOIN intercambiando el orden de las tablas:

$$A \text{ RIGHT JOIN } B \equiv B \text{ LEFT JOIN } A$$

6.4. UNION**Definición: UNION**

Combina verticalmente los resultados de dos o más sentencias SELECT. UNION elimina filas duplicadas; UNION ALL las conserva. Ambas consultas deben tener el mismo número de columnas y tipos de datos compatibles.

Sintaxis general:

```

1 SELECT columna1, columna2, ...

```

```
2 FROM tabla_a
3 UNION [ALL]
4 SELECT columna1, columna2, ...
5 FROM tabla_b;
```

Listing 21: Sintaxis de UNION / UNION ALL

Ejemplo práctico

Obtener una lista unificada de correos de dos tablas distintas (clientes y proveedores), sin duplicados:

```
1 SELECT nombre, correo, 'cliente' AS origen
2 FROM clientes
3 UNION
4 SELECT nombre, correo, 'proveedor' AS origen
5 FROM proveedores
6 ORDER BY nombre;
```

Listing 22: UNION de correos de clientes y proveedores

Ejemplo práctico

Comparación entre UNION y UNION ALL:

```
1 -- Elimina duplicados (m s lento: realiza ORDER +
   DISTINCT internamente)
2 SELECT ciudad FROM clientes
3 UNION
4 SELECT ciudad FROM proveedores;
5
6 -- Conserva duplicados (m s r pido: no realiza
   deduplicación)
7 SELECT ciudad FROM clientes
8 UNION ALL
9 SELECT ciudad FROM proveedores;
```

Listing 23: Diferencia entre UNION y UNION ALL

Nota importante

Los nombres de las columnas del resultado se toman del **primer SELECT**. La cláusula ORDER BY solo puede aparecer **una vez**, al final de toda la sentencia, y afecta al conjunto combinado completo.

Resumen comparativo

Tabla 8: Comparación de operaciones de combinación en SQL

Operación	Filas incluidas	Uso típico
INNER JOIN	Solo las que tienen coincidencia en ambas tablas	Datos relacionados que siempre existen
LEFT JOIN	Todas las de la tabla izquierda + coincidencias	Encontrar registros sin par (huérfanos)
RIGHT JOIN	Todas las de la tabla derecha + coincidencias	Simétrico de LEFT JOIN
UNION	Filas de ambos SELECT, sin duplicados	Consolidar datos de tablas similares
UNION ALL	Filas de ambos SELECT, con duplicados	Igual que UNION pero más eficiente

7. Modificadores: Parte 1

Los **modificadores** son cláusulas y operadores que se combinan con **SELECT** para refinar, filtrar y ordenar los resultados de una consulta. En esta sección cubrimos los seis modificadores fundamentales que todo programador SQL debe dominar desde el primer día. Todos los ejemplos siguen usando la tabla **empleados** introducida en la sección anterior.

7.1. DISTINCT

Por defecto, **SELECT** devuelve **todas** las filas que satisfacen la consulta, incluyendo duplicados. La palabra clave **DISTINCT** instruye al SGBD a eliminar las filas repetidas del resultado, conservando sólo valores únicos.

Definición: DISTINCT

Modificador que se coloca inmediatamente después de **SELECT** y elimina del resultado todas las filas cuyo conjunto de valores en las columnas proyectadas sea idéntico a otra fila ya presente.

```

1  -- Qu departamentos existen en la empresa?
2  -- Sin DISTINCT aparecer an 10 filas (una por empleado)
3  SELECT DISTINCT departamento
4  FROM    empleados;
5
6  -- Resultado:
7  -- Ventas
8  -- TI
9  -- RRHH

```

```
10 -- Finanzas
```

Listing 24: DISTINCT sobre una columna.

```
1 -- Combinaciones nicas de departamento + cargo
2 SELECT DISTINCT departamento, cargo
3 FROM empleados
4 ORDER BY departamento, cargo;
```

Listing 25: DISTINCT sobre múltiples columnas.

Nota importante

DISTINCT opera sobre la *combinación completa* de todas las columnas proyectadas, no columna por columna. En el ejemplo anterior, ('TI', 'Desarrolladora') es una combinación única aunque haya dos desarrolladoras en TI, porque ambas tienen exactamente los mismos valores en esas dos columnas — en ese caso sí se eliminará el duplicado.

7.2. WHERE

La cláusula WHERE es el mecanismo de **filtrado por fila** de SQL. Evalúa una condición booleana para cada fila de la tabla fuente; sólo las filas donde la condición resulta TRUE pasan al resultado.

Definición: WHERE

Cláusula que restringe las filas devueltas por una consulta a aquellas que satisfacen una expresión booleana dada. Se evalúa *antes* de GROUP BY, HAVING y SELECT.

```
1 -- Empleados del departamento de TI
2 SELECT nombre, cargo, salario
3 FROM empleados
4 WHERE departamento = 'TI';
5
6 -- Empleados con salario mayor a 45,000
7 SELECT nombre, salario
8 FROM empleados
9 WHERE salario > 45000;
10
11 -- Empleados que ingresaron despu s del 1 de enero de 2020
12 SELECT nombre, fecha_ingreso
13 FROM empleados
14 WHERE fecha_ingreso > '2020-01-01';
```

Listing 26: Filtros básicos con WHERE.

Ejemplo práctico

Imagina la tabla como un tamiz: cada fila cae por los agujeros a menos que cumpla la condición de **WHERE**. Las que no la cumplen simplemente desaparecen del resultado. El dato original en la tabla **no se modifica**; sólo se filtra lo que se muestra.

7.3. ORDER BY

ORDER BY define el **criterio de ordenamiento** del resultado final. Sin él, el SGBD no garantiza ningún orden particular entre ejecuciones.

```
1  -- Orden ascendente (A      Z, menor      mayor): por defecto
2  SELECT nombre, salario
3  FROM    empleados
4  ORDER BY salario ASC;    -- ASC es opcional, es el valor por
                           defecto
5
6  -- Orden descendente (mayor      menor)
7  SELECT nombre, salario
8  FROM    empleados
9  ORDER BY salario DESC;
```

Listing 27: Orden ascendente y descendente.

```
1  -- Primero por departamento ( A Z ),
2  -- dentro de cada departamento por salario ( m a y o r menor )
3  SELECT nombre, departamento, salario
4  FROM    empleados
5  ORDER BY departamento ASC,
6          salario      DESC;
```

Listing 28: Ordenamiento compuesto por múltiples columnas.

```
1  -- Se puede referenciar la columna por su posici n en
   SELECT
2  -- (poco recomendado: hace el c digo menos legible)
3  SELECT nombre, departamento, salario
4  FROM    empleados
5  ORDER BY 3 DESC;    -- 3 = tercera columna = salario
```

Listing 29: ORDER BY usando posición numérica de columna.

Nota importante

El uso de posición numérica en **ORDER BY** está permitido pero se desaconseja en código de producción: si el orden de las columnas en **SELECT** cambia, el ordenamiento cambia sin ningún aviso de error.

7.4. LIKE

LIKE permite comparar una columna de texto contra un **patrón**, usando caracteres comodín. Es la herramienta de búsqueda de texto parcial más básica de SQL.

Comodín	Significado	Ejemplo
%	Cero o más caracteres cualesquiera	'Ana%' coincide con Ana, Analista, Anabel
_	Exactamente un carácter cualquiera	'_na' coincide con Ana, Ina, Una

Tabla 9: Comodines del operador LIKE.

```

1  -- Nombres que empiezan con 'A'
2  SELECT nombre FROM empleados
3  WHERE nombre LIKE 'A%';
4
5  -- Nombres que terminan con 'ez'
6  SELECT nombre FROM empleados
7  WHERE nombre LIKE '%ez';
8
9  -- Nombres que contienen 'ar' en cualquier posición
10 SELECT nombre FROM empleados
11 WHERE nombre LIKE '%ar%';
12
13 -- Nombres cuya segunda letra es 'o'
14 SELECT nombre FROM empleados
15 WHERE nombre LIKE '_o%';
16
17 -- Nombres que NO contienen 'a' (NOT LIKE)
18 SELECT nombre FROM empleados
19 WHERE nombre NOT LIKE '%a%';

```

Listing 30: Patrones con LIKE.

Nota importante

En MySQL, LIKE es **insensible a mayúsculas/minúsculas** por defecto cuando la columna usa un *collation* insensible (utf8_general_ci, utf8mb4_unicode_ci, etc.). Si necesitas distinguir mayúsculas usa LIKE BINARY o cambia el collation de la columna.

7.5. AND, OR, NOT

Los operadores lógicos permiten **combinar múltiples condiciones** dentro de una cláusula WHERE.

Operador	Resultado TRUE cuando...	Precedencia
NOT	La condición que sigue es FALSE	Mayor (se evalúa primero)
AND	Ambas condiciones son TRUE	Media
OR	Al menos una condición es TRUE	Menor (se evalúa al final)

Tabla 10: Operadores lógicos y su precedencia.

```

1  -- AND: empleados de TI con salario mayor a 47,000
2  SELECT nombre, departamento, salario
3  FROM    empleados
4  WHERE   departamento = 'TI'
5         AND   salario > 47000;
6
7  -- OR: empleados de Ventas O de Finanzas
8  SELECT nombre, departamento
9  FROM    empleados
10 WHERE   departamento = 'Ventas'
11        OR   departamento = 'Finanzas';
12
13 -- NOT: todos los departamentos excepto RRHH
14 SELECT nombre, departamento
15 FROM    empleados
16 WHERE   NOT departamento = 'RRHH';
17 -- Equivalente: WHERE departamento <> 'RRHH'

```

Listing 31: Uso de AND, OR y NOT.

```

1  -- Gerentes de TI O analistas de Ventas con salario >
   30,000
2  -- Las par ntesis son ESENCIALES aqu
3  SELECT nombre, departamento, cargo, salario
4  FROM    empleados
5  WHERE   (departamento = 'TI'      AND cargo = 'Gerente')
6         OR   (departamento = 'Ventas' AND cargo = 'Analista'
7              AND salario > 30000);

```

Listing 32: Combinación de operadores y uso de paréntesis.

Nota importante

AND tiene mayor precedencia que OR, igual que la multiplicación tiene mayor precedencia que la suma en aritmética. Sin paréntesis, A OR B AND C se interpreta como A OR (B AND C), lo que puede producir resultados inesperados. **Usa siempre paréntesis** cuando combines AND y OR en la misma condición.

7.6. LIMIT

LIMIT restringe el número máximo de filas que la consulta devuelve. Combinado con OFFSET (o la sintaxis abreviada de MySQL) permite implementar **paginación**.

```
1  -- Los 5 empleados con mayor salario
2  SELECT nombre, salario
3  FROM    empleados
4  ORDER BY salario DESC
5  LIMIT 5;
```

Listing 33: LIMIT básico.

```
1  -- P g i n a 1: primeras 3 filas (OFFSET 0, impl cito)
2  SELECT nombre, departamento
3  FROM    empleados
4  ORDER BY nombre ASC
5  LIMIT 3;
6
7  -- P g i n a 2: filas 4, 5 y 6
8  SELECT nombre, departamento
9  FROM    empleados
10 ORDER BY nombre ASC
11 LIMIT 3 OFFSET 3;
12
13 -- P g i n a 3: filas 7, 8 y 9
14 SELECT nombre, departamento
15 FROM    empleados
16 ORDER BY nombre ASC
17 LIMIT 3 OFFSET 6;
```

Listing 34: LIMIT con OFFSET para paginación.

```
1  -- Equivalente a LIMIT 3 OFFSET 6
2  SELECT nombre, departamento
3  FROM    empleados
4  ORDER BY nombre ASC
5  LIMIT 6, 3;    -- primero el offset, luego la cantidad
```

Listing 35: Sintaxis abreviada MySQL: LIMIT offset, cantidad.

Ejemplo práctico

La paginación es imprescindible en aplicaciones web. Si una tabla tiene 500,000 registros y una página muestra 20, la consulta para la página 3 sería: LIMIT 20 OFFSET 40. Sin LIMIT, traer 500,000 filas al navegador colapsaría tanto la base de datos como la red.

Nota importante

LIMIT sin ORDER BY produce resultados **no deterministas**: en dos ejecuciones distintas podrían devolverse filas diferentes. Siempre acompáñalo de ORDER BY para obtener páginas consistentes.

8. Modificadores: Parte 2

En esta sección cubrimos los modificadores avanzados de consulta: funciones de agregación, operadores de rango y lista, alias, concatenación, agrupamiento, lógica condicional y manejo de nulos. Todos son componentes esenciales para construir consultas analíticas y reportes sobre los datos.

8.1. Valores NULL

NULL no es un valor como 0 o ""; es la **ausencia de valor**. Representa información desconocida o no aplicable, y su comportamiento en operaciones es especial: cualquier operación aritmética o comparación con NULL devuelve NULL (ni TRUE ni FALSE).

Definición: NULL

Marcador especial que indica la ausencia de un valor en una columna. No es equivalente a cero, a una cadena vacía ni a FALSE. Para comprobar si un valor es NULL se usan los operadores IS NULL e IS NOT NULL; el operador = nunca devuelve TRUE al comparar con NULL.

```
1  -- INCORRECTO: esta condici n nunca devuelve filas
2  -- porque NULL = NULL es NULL (no TRUE)
3  SELECT * FROM empleados WHERE telefono = NULL;    -- MAL !
4
5  -- CORRECTO: usar IS NULL
6  SELECT * FROM empleados WHERE telefono IS NULL;
7
8
9  -- Empleados que S  tienen tel fono registrado
10 SELECT * FROM empleados WHERE telefono IS NOT NULL;
11
12 -- NULL en aritm tica: el resultado es siempre NULL
13 SELECT nombre, salario + NULL AS resultado --      NULL
14 FROM     empleados;
15
16 -- NULL en ORDER BY: MySQL los coloca al INICIO en ASC
17 -- y al FINAL en DESC (var a entre SGBD)
18 SELECT nombre, telefono
19 FROM     empleados
20 ORDER BY telefono ASC;
```

Listing 36: Comportamiento de NULL y cómo detectarlo.

Ejemplo práctico

Supón que agregas una columna `telefono` a la tabla `empleados` sin indicar NOT NULL. Los registros existentes quedarán con NULL en esa columna, no con 0 ni con ". Si después calculas `COUNT(telefono)`, esos registros *no* se contarán, porque COUNT ignora NULLs.

8.2. MIN y MAX

MIN y MAX son **funciones de agregación** que devuelven el valor mínimo y máximo de una columna dentro de un conjunto de filas. Funcionan sobre números, fechas y cadenas de texto.

```
1  -- Salario m s bajo y m s alto de toda la empresa
2  SELECT MIN(salario) AS salario_minimo,
3         MAX(salario) AS salario_maximo
4  FROM    empleados;
5
6  -- MIN y MAX por departamento
7  SELECT departamento,
8         MIN(salario) AS minimo,
9         MAX(salario) AS maximo,
10        MAX(salario) - MIN(salario) AS rango_salarial
11 FROM    empleados
12 GROUP BY departamento
13 ORDER BY rango_salarial DESC;
14
15 -- Fecha de ingreso m s antigua y m s reciente
16 SELECT MIN(fecha_ingreso) AS primer_ingreso,
17        MAX(fecha_ingreso) AS ultimo_ingreso
18 FROM    empleados;
```

Listing 37: MIN y MAX sobre números y fechas.

Nota importante

MIN y MAX **ignoran** los valores NULL automáticamente. Si todas las filas de un grupo son NULL, el resultado de la función es NULL.

8.3. COUNT

COUNT cuenta filas o valores no nulos. Tiene tres variantes con comportamientos distintos:

```
1  -- COUNT(*): cuenta TODAS las filas, incluyendo NULLs
2  SELECT COUNT(*) AS total_empleados
3  FROM    empleados;
4  -- Resultado: 10
5
6  -- COUNT(columna): cuenta s lo las filas donde
7  -- esa columna NO es NULL
8  SELECT COUNT(telefono) AS empleados_con_telefono
9  FROM    empleados;
10 -- Si 3 tienen NULL en telefono      resultado: 7
11
12 -- COUNT(DISTINCT columna): cuenta valores nicos no nulos
13 SELECT COUNT(DISTINCT departamento) AS num_departamentos
14 FROM    empleados;
```

```
15 -- Resultado: 4
16
17 -- COUNT con GROUP BY: total por grupo
18 SELECT departamento, COUNT(*) AS num_empleados
19 FROM empleados
20 GROUP BY departamento
21 ORDER BY num_empleados DESC;
```

Listing 38: Variantes de COUNT.

8.4. SUM

SUM suma todos los valores no nulos de una columna numérica dentro del conjunto de filas evaluado.

```
1 -- Masa salarial total de la empresa
2 SELECT SUM(salario) AS masa_salarial_total
3 FROM empleados;
4
5 -- Masa salarial por departamento
6 SELECT departamento,
7     SUM(salario) AS masa_salarial,
8     COUNT(*) AS num_empleados
9 FROM empleados
10 GROUP BY departamento
11 ORDER BY masa_salarial DESC;
12
13 -- SUM condicional usando CASE (suma s lo gerentes)
14 SELECT SUM(CASE WHEN cargo = 'Gerente'
15     THEN salario ELSE 0 END) AS total_gerentes
16 FROM empleados;
```

Listing 39: SUM: suma total y por grupo.

8.5. AVG

AVG calcula el **promedio aritmético** de los valores no nulos de una columna numérica. Equivale a $\text{SUM}(\text{col}) / \text{COUNT}(\text{col})$, pero ignorando NULLs en ambos términos.

```
1 -- Salario promedio de toda la empresa
2 SELECT ROUND(AVG(salario), 2) AS salario_promedio
3 FROM empleados;
4
5 -- Promedio por departamento, con indicador sobre/bajo
   media global
6 SELECT departamento,
7     ROUND(AVG(salario), 2) AS promedio_dpto,
8     ROUND(AVG(salario) - (SELECT AVG(salario)
9         FROM empleados), 2) AS
10     diferencia_global
11 FROM empleados
```

```
11 GROUP BY departamento
12 ORDER BY promedio_dpto DESC;
```

Listing 40: AVG: promedio general y por grupo.

Nota importante

AVG ignora los NULLs: si una columna tiene 10 filas pero 3 son NULL, el promedio se calcula sobre los 7 valores restantes, no dividiendo entre 10. Si quieres tratar los NULLs como cero, usa `AVG(COALESCE(columna, 0))`.

8.6. IN

IN verifica si el valor de una columna pertenece a una **lista de valores** especificada. Es equivalente a una cadena de condiciones OR sobre la misma columna, pero más legible y compacto.

```
1  -- Empleados de Ventas, TI o Finanzas
2  -- Equivalente: WHERE departamento = 'Ventas'
3  --               OR departamento = 'TI'
4  --               OR departamento = 'Finanzas'
5  SELECT nombre, departamento
6  FROM    empleados
7  WHERE   departamento IN ('Ventas', 'TI', 'Finanzas');
8
9  -- NOT IN: departamentos excluidos
10 SELECT nombre, departamento
11 FROM    empleados
12 WHERE   departamento NOT IN ('RRHH');
13
14 -- IN con subconsulta: departamentos que tienen gerente
15 SELECT nombre, departamento
16 FROM    empleados
17 WHERE   departamento IN (
18     SELECT DISTINCT departamento
19     FROM    empleados
20     WHERE   cargo = 'Gerente'
21 );
```

Listing 41: IN con lista de valores literales.

Nota importante

Precaución con NOT IN y NULLs: si la lista de NOT IN contiene un NULL, la condición completa devuelve NULL (ninguna fila), porque SQL no puede determinar si el valor es distinto de algo desconocido. En subconsultas, usa NOT EXISTS en lugar de NOT IN si los resultados pueden contener NULLs.

8.7. BETWEEN

BETWEEN filtra filas cuyos valores se encuentran dentro de un **rango inclusivo** (incluye los extremos). Funciona con números, fechas y cadenas.

```

1  -- Empleados con salario entre 35,000 y 52,000 (inclusive)
2  -- Equivalente: WHERE salario >= 35000 AND salario <= 52000
3  SELECT nombre, salario
4  FROM    empleados
5  WHERE   salario BETWEEN 35000 AND 52000;
6
7  -- NOT BETWEEN: fuera del rango
8  SELECT nombre, salario
9  FROM    empleados
10 WHERE   salario NOT BETWEEN 35000 AND 52000;
11
12 -- BETWEEN con fechas
13 SELECT nombre, fecha_ingreso
14 FROM    empleados
15 WHERE   fecha_ingreso BETWEEN '2019-01-01' AND '2021-12-31';
16
17 -- BETWEEN con cadenas (orden alfabético)
18 SELECT nombre
19 FROM    empleados
20 WHERE   nombre BETWEEN 'C' AND 'M'; -- nombres de C a M

```

Listing 42: BETWEEN con números y fechas.

8.8. ALIAS (AS)

Un **alias** es un nombre temporal que se asigna a una columna o tabla dentro de una consulta. Mejora la legibilidad del resultado y es indispensable al trabajar con expresiones calculadas.

```

1  SELECT
2      nombre                AS empleado,
3      salario               AS sueldo_base,
4      salario * 12          AS ingreso_anual,
5      ROUND(salario / 30, 2) AS sueldo_diario,
6      CONCAT(nombre, ' (', cargo, ')') AS descripcion
7  FROM empleados;

```

Listing 43: Alias de columna con AS.

```

1  -- Alias de tabla: simplifica nombres largos
2  SELECT e.nombre, e.departamento, e.salario
3  FROM    empleados AS e
4  WHERE   e.salario > 40000;
5
6  -- La palabra AS es opcional (funciona igual sin ella)
7  SELECT e.nombre
8  FROM    empleados e

```

```
9 WHERE e.cargo = 'Gerente';
```

Listing 44: Alias de tabla: útil con JOINS y subconsultas.

Nota importante

Los alias de columna definidos en **SELECT** **no pueden usarse** en la cláusula **WHERE** (porque **WHERE** se evalúa antes que **SELECT**). Sí pueden usarse en **ORDER BY** y **HAVING** en MySQL, aunque esto no es estándar en todos los SGBD.

8.9. CONCAT

CONCAT une dos o más cadenas de texto en una sola. Es una función (no un operador) y acepta cualquier número de argumentos.

```
1 -- Nombre completo con cargo entre parntesis
2 SELECT CONCAT(nombre, ' ', cargo) AS presentacion
3 FROM empleados;
4
5 -- CONCAT con separador usando CONCAT_WS
6 -- (WithSeparator): el primer argumento es el separador
7 SELECT CONCAT_WS(' ', nombre, departamento, cargo) AS
8     ficha
9 FROM empleados;
10
11 -- Combinar texto literal con valores de columnas
12 SELECT CONCAT('El empleado ', nombre,
13     ' gana $', FORMAT(salario, 2),
14     ' al mes.') AS resumen
15 FROM empleados
16 WHERE cargo = 'Gerente';
```

Listing 45: **CONCAT** para construir cadenas compuestas.

Nota importante

En MySQL, si **cualquier** argumento de **CONCAT** es **NULL**, el resultado completo es **NULL**. Para evitarlo, usa **COALESCE**:
CONCAT(COALESCE(nombre, 'Sin nombre'), ' - ', cargo). **CONCAT_WS** es más tolerante: ignora los argumentos **NULL** en lugar de devolver **NULL**.

8.10. GROUP BY

GROUP BY agrupa las filas que tienen valores idénticos en las columnas especificadas, para que las funciones de agregación (**COUNT**, **SUM**, **AVG**, etc.) operen sobre cada grupo por separado en lugar de sobre toda la tabla.

Definición: GROUP BY

Cláusula que divide las filas del resultado de **WHERE** en grupos, donde cada grupo comparte el mismo valor en las columnas de agrupamiento. Cada grupo produce exactamente **una fila** en el resultado.

```

1  -- Número de empleados y salario promedio por departamento
2  SELECT departamento,
3         COUNT(*)           AS num_empleados,
4         ROUND(AVG(salario), 2) AS promedio,
5         SUM(salario)       AS masa_salarial,
6         MIN(salario)       AS minimo,
7         MAX(salario)       AS maximo
8  FROM    empleados
9  GROUP BY departamento
10 ORDER BY masa_salarial DESC;
```

Listing 46: GROUP BY con funciones de agregación.

```

1  -- Combinaciones únicas de departamento + cargo con conteo
2  SELECT departamento,
3         cargo,
4         COUNT(*) AS cantidad
5  FROM    empleados
6  GROUP BY departamento, cargo
7  ORDER BY departamento, cargo;
```

Listing 47: GROUP BY con múltiples columnas.

Nota importante

En SQL estándar (y en MySQL con `ONLY_FULL_GROUP_BY` activado), **toda columna** que aparezca en **SELECT** y no esté dentro de una función de agregación debe estar en **GROUP BY**. Si no, el SGBD no sabe qué valor mostrar de entre las múltiples filas del grupo.

8.11. HAVING

HAVING filtra **grupos** (no filas individuales). Es el equivalente de **WHERE** pero aplicado después de que **GROUP BY** ha formado los grupos.

Definición: HAVING

Cláusula que filtra los grupos producidos por **GROUP BY**, conservando sólo aquellos cuya condición (generalmente sobre una función de agregación) es **TRUE**.

```

1  -- Departamentos donde el salario promedio supera 45,000
2  SELECT departamento,
3         ROUND(AVG(salario), 2) AS promedio
4  FROM    empleados
```

```

5 GROUP BY departamento
6 HAVING AVG(salario) > 45000;
7
8 -- Departamentos con m s de 2 empleados
9 SELECT departamento,
10      COUNT(*) AS num_empleados
11 FROM empleados
12 GROUP BY departamento
13 HAVING COUNT(*) > 2;
14
15 -- Combinando WHERE (filtra filas) y HAVING (filtra grupos)
16 -- Departamentos con m s de 1 analista
17 SELECT departamento,
18      COUNT(*) AS num_analistas
19 FROM empleados
20 WHERE cargo = 'Analista'           -- primero filtra por
   cargo
21 GROUP BY departamento
22 HAVING COUNT(*) > 1;               -- luego filtra grupos

```

Listing 48: HAVING: filtrar grupos por condición agregada.

Ejemplo práctico

La diferencia entre **WHERE** y **HAVING** puede resumirse así: **WHERE** descarta filas *antes* de agrupar (actúa sobre datos individuales); **HAVING** descarta grupos *después* de agrupar (actúa sobre resultados de agregación). Intentar usar una función de agregación en **WHERE** produce un error de sintaxis.

8.12. CASE

CASE es la expresión de **lógica condicional** de SQL: permite evaluar condiciones dentro de una consulta y devolver valores diferentes según cuál se cumpla. Equivale al **if/else** o al **switch** de los lenguajes de programación.

Existen dos formas sintácticas:

```

1 -- Traduce el nombre del departamento al inglés
2 SELECT nombre,
3      CASE departamento
4          WHEN 'Ventas' THEN 'Sales'
5          WHEN 'TI' THEN 'IT'
6          WHEN 'RRHH' THEN 'HR'
7          WHEN 'Finanzas' THEN 'Finance'
8          ELSE 'Unknown'
9      END AS department_en
10 FROM empleados;

```

Listing 49: CASE simple (compara un valor contra una lista).

```

1 -- Clasifica el salario en bandas
2 SELECT nombre,

```

```

3      salario ,
4      CASE
5          WHEN salario >= 55000 THEN 'Banda A (Senior)'
6          WHEN salario >= 45000 THEN 'Banda B (Mid)'
7          WHEN salario >= 35000 THEN 'Banda C (Junior)'
8          ELSE 'Banda D (Entrada)'
9      END AS banda_salarial
10 FROM empleados
11 ORDER BY salario DESC;

```

Listing 50: CASE buscado (evalúa condiciones booleanas).

```

1  -- Cuenta cu ntos empleados hay en cada banda salarial
2  SELECT
3      COUNT(CASE WHEN salario >= 55000 THEN 1 END) AS banda_a
4      ,
5      COUNT(CASE WHEN salario BETWEEN 45000 AND 54999 THEN 1
6              END) AS banda_b,
7      COUNT(CASE WHEN salario BETWEEN 35000 AND 44999 THEN 1
8              END) AS banda_c,
9      COUNT(CASE WHEN salario < 35000 THEN 1 END) AS banda_d
10 FROM empleados;

```

Listing 51: CASE dentro de funciones de agregación.

8.13. IFNULL y COALESCE

Ambas funciones permiten sustituir un valor NULL por un valor alternativo. Son esenciales para producir resultados legibles cuando los datos pueden ser incompletos.

```

1  -- Si telefono es NULL, muestra 'Sin registro'
2  SELECT nombre,
3          IFNULL(telefono, 'Sin registro') AS telefono
4  FROM empleados;
5
6  -- IFNULL en c lculos: evita que NULL infecte el resultado
7  SELECT nombre,
8          salario + IFNULL(bono, 0) AS salario_total
9  FROM empleados;

```

Listing 52: IFNULL: sustituye NULL por un valor por defecto.

```

1  -- Devuelve el primer contacto disponible entre tres
   opciones
2  SELECT nombre,
3          COALESCE(telefono_celular,
4                  telefono_fijo,
5                  email,
6                  'Sin contacto') AS contacto_preferido
7  FROM empleados;

```

Listing 53: COALESCE: devuelve el primer valor no NULL de una lista.

Nota importante

IFNULL(a, b) es equivalente a COALESCE(a, b), pero COALESCE acepta cualquier número de argumentos y es parte del estándar SQL. Prefiere COALESCE para mayor portabilidad entre SGBD.

8.14. Otros Modificadores

8.14.1. DISTINCT con funciones de agregación

DISTINCT puede usarse *dentro* de funciones de agregación para contar o sumar solo valores únicos:

```

1  -- Cu ntos cargos diferentes existen en la empresa
2  SELECT COUNT(DISTINCT cargo) AS cargos_distintos
3  FROM    empleados;
4
5  -- Suma de salarios  nicos  (ignora repetidos)
6  SELECT SUM(DISTINCT salario) AS suma_salarios_unicos
7  FROM    empleados;
```

Listing 54: DISTINCT dentro de funciones de agregación.

8.14.2. ROLLUP: subtotales y total general

WITH ROLLUP agrega filas extra de subtotales a un resultado de GROUP BY:

```

1  -- Masa salarial por departamento + total general al final
2  SELECT
3      COALESCE(departamento, 'TOTAL EMPRESA') AS nivel,
4      COUNT(*)      AS empleados,
5      SUM(salario)  AS masa_salarial
6  FROM    empleados
7  GROUP BY departamento WITH ROLLUP;
8  -- La ltima  fila tendr  NULL en departamento (el gran
   total)
9  -- COALESCE lo convierte en 'TOTAL EMPRESA'
```

Listing 55: GROUP BY con ROLLUP para subtotales.

8.14.3. Expresiones aritméticas en GROUP BY y HAVING

```

1  -- A o de ingreso como criterio de agrupamiento
2  SELECT YEAR(fecha_ingreso) AS anio,
3      COUNT(*)              AS contrataciones,
4      SUM(salario)          AS masa_salarial_ese_anio
5  FROM    empleados
6  GROUP BY YEAR(fecha_ingreso)
7  HAVING COUNT(*) >= 2
8  ORDER BY anio DESC;
```

Listing 56: Expresiones calculadas combinadas con agrupamiento.

8.14.4. FORMAT: formato de números en la salida

```
1  -- Salarios con separador de miles y 2 decimales
2  SELECT nombre,
3         FORMAT(salario, 2)      AS salario_fmt,      --
4         '48,000.00'
5         FORMAT(salario * 12, 0)  AS anual_fmt        --
6         '576,000'
7  FROM    empleados
8  ORDER BY salario DESC;
```

Listing 57: FORMAT para presentación de números.

9. Almacenamiento de Datos Relacionados

Cuando una base de datos posee **integridad referencial** activa, el orden y la forma en que se insertan los registros importan tanto como el contenido mismo. Violar el orden de inserción provoca errores de clave foránea que el motor rechaza antes de escribir cualquier dato.

Nota importante

Regla general: insertar siempre primero la tabla **padre** (la que contiene la clave primaria referenciada) y después la tabla **hijo** (la que contiene la clave foránea). Para eliminar, el orden es el inverso.

9.1. Inserción en Tablas con Relación 1:1

Definición: Relación 1:1

Una fila de la tabla **A** se corresponde con **exactamente una** fila de la tabla **B**, y viceversa. Se implementa colocando la clave foránea en la tabla secundaria con restricción **UNIQUE**.

Esquema de ejemplo:

```

1 CREATE TABLE empleados (
2     empleado_id INT UNSIGNED AUTO_INCREMENT
3     PRIMARY KEY,
4     nombre VARCHAR(100) NOT NULL,
5     departamento VARCHAR(60) NOT NULL
6 );
7
8 CREATE TABLE expedientes (
9     expediente_id INT UNSIGNED AUTO_INCREMENT
10    PRIMARY KEY,
11    empleado_id INT UNSIGNED NOT NULL UNIQUE,
12    -- UNIQUE fuerza 1:1
13    nss CHAR(11) NOT NULL,
14    fecha_alta DATE NOT NULL,
15    FOREIGN KEY (empleado_id) REFERENCES empleados(
16        empleado_id)
17    ON DELETE CASCADE
18    ON UPDATE CASCADE
19 );

```

Listing 58: Tablas con relación 1:1 — empleado y expediente

Orden correcto de inserción:

```

1 -- Paso 1: insertar en la tabla PADRE
2 INSERT INTO empleados (nombre, departamento)
3 VALUES ('Laura Mendoza', 'Ingenier a'),
4         ('Roberto Solis', 'Finanzas');
5

```

```

6  -- Paso 2: recuperar los IDs generados (o usar
   LAST_INSERT_ID() para uno solo)
7  -- empleado_id = 1 -> Laura Mendoza
8  -- empleado_id = 2 -> Roberto Solis
9
10 -- Paso 3: insertar en la tabla HIJO referenciando los IDs
   existentes
11 INSERT INTO expedientes (empleado_id, nss, fecha_alta)
12 VALUES (1, '578-12-3456', '2021-03-15'),
13          (2, '412-87-9900', '2019-07-01');

```

Listing 59: Inserción respetando integridad referencial 1:1

Ejemplo práctico

Uso de LAST_INSERT_ID() para encadenar inserciones individuales de forma segura:

```

1  INSERT INTO empleados (nombre, departamento)
2  VALUES ('Carmen Vidal', 'Diseño');
3
4  -- LAST_INSERT_ID() devuelve el AUTO_INCREMENT de la
   ltima inserción
5  INSERT INTO expedientes (empleado_id, nss, fecha_alta)
6  VALUES (LAST_INSERT_ID(), '301-55-7812', '2023-01-10');

```

Listing 60: Encadenamiento con LAST_INSERT_ID()

Nota importante

LAST_INSERT_ID() es **seguro en entornos concurrentes**: cada conexión al servidor mantiene su propio contador, por lo que dos sesiones simultáneas no interfieren entre sí.

9.2. Inserción en Tablas con Relación 1:N

Definición: Relación 1:N

Una fila de la tabla **padre** puede asociarse con **múltiples** filas de la tabla **hijo**, pero cada fila hijo pertenece a un único padre. Es la cardinalidad más frecuente en bases de datos relacionales.

Esquema de ejemplo:

```

1  CREATE TABLE clientes (
2      cliente_id INT UNSIGNED AUTO_INCREMENT
   PRIMARY KEY,
3      nombre VARCHAR(100) NOT NULL,
4      correo VARCHAR(120) NOT NULL UNIQUE
5  );
6

```

```
7 CREATE TABLE pedidos (
8     pedido_id INT UNSIGNED AUTO_INCREMENT
9     PRIMARY KEY,
10    cliente_id INT UNSIGNED NOT NULL,
11    fecha DATE NOT NULL,
12    total DECIMAL(10,2) NOT NULL DEFAULT 0.00,
13    FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id)
14    ON DELETE RESTRICT
15    ON UPDATE CASCADE
16 );
```

Listing 61: Tablas con relación 1:N — cliente y pedidos

Inserción de registros dependientes:

```
1 -- Paso 1: insertar el cliente (tabla padre)
2 INSERT INTO clientes (nombre, correo)
3 VALUES ('Ana Torres', 'ana.torres@correo.com');
4
5 -- Paso 2: insertar sus pedidos (tabla hijo)
6 --       Todos referencian el mismo cliente_id
7 INSERT INTO pedidos (cliente_id, fecha, total)
8 VALUES
9     (LAST_INSERT_ID(), '2024-01-15', 1250.00),
10    (LAST_INSERT_ID(), '2024-03-02', 340.50),
11    (LAST_INSERT_ID(), '2024-06-18', 2890.75);
```

Listing 62: Inserción 1:N — un cliente con varios pedidos

Ejemplo práctico

Inserción de pedidos para un cliente ya existente, identificado por correo:

```
1 INSERT INTO pedidos (cliente_id, fecha, total)
2 SELECT cliente_id, '2024-09-10', 780.00
3 FROM clientes
4 WHERE correo = 'ana.torres@correo.com';
```

Listing 63: INSERT ... SELECT para referenciar por atributo

Este patrón evita tener que conocer el `cliente_id` numérico de antemano y es útil en scripts de carga masiva.

Nota importante

La política `ON DELETE RESTRICT` impide eliminar un cliente mientras tenga pedidos asociados. Para poder borrarlo habría que eliminar primero sus pedidos, o bien cambiar la política a `ON DELETE CASCADE`.

9.3. Inserción en Tablas con Relación N:M

Definición: Relación N:M

Múltiples filas de la tabla **A** pueden relacionarse con múltiples filas de la tabla **B**. Se implementa mediante una **tabla intermedia** (también llamada tabla de unión o pivote) que contiene, al menos, las claves foráneas de ambas entidades.

Esquema de ejemplo:

```

1 CREATE TABLE estudiantes (
2     estudiante_id INT UNSIGNED AUTO_INCREMENT
3     PRIMARY KEY,
4     nombre VARCHAR(100) NOT NULL
5 );
6
7 CREATE TABLE cursos (
8     curso_id INT UNSIGNED AUTO_INCREMENT
9     PRIMARY KEY,
10    titulo VARCHAR(120) NOT NULL,
11    creditos TINYINT UNSIGNED NOT NULL
12 );
13
14 -- Tabla intermedia: un estudiante puede tomar N cursos
15 -- un curso puede tener N estudiantes
16
17 CREATE TABLE inscripciones (
18     estudiante_id INT UNSIGNED NOT NULL,
19     curso_id INT UNSIGNED NOT NULL,
20     fecha_alta DATE NOT NULL,
21     calificacion DECIMAL(4,2) DEFAULT NULL,
22     PRIMARY KEY (estudiante_id, curso_id), -- PK
23     compuesta evita duplicados
24     FOREIGN KEY (estudiante_id) REFERENCES estudiantes(
25         estudiante_id)
26     ON DELETE CASCADE ON UPDATE CASCADE,
27     FOREIGN KEY (curso_id) REFERENCES cursos(curso_id)
28     ON DELETE CASCADE ON UPDATE CASCADE
29 );

```

Listing 64: Tablas con relación N:M — estudiantes y cursos

Orden de inserción en tres pasos:

```

1 -- Tabla padre A
2 INSERT INTO estudiantes (nombre)
3 VALUES ('Miguel R os'),
4         ('Sof a Castillo'),
5         ('H ctor N ez');
6
7 -- Tabla padre B
8 INSERT INTO cursos (titulo, creditos)
9 VALUES ('Bases de Datos', 6),
10        (' lgebra Lineal', 4),

```

```
11 ('Redes de C mputo', 5);
```

Listing 65: Paso 1 — poblar ambas tablas padre

```
1 -- IDs resultantes (ejemplo):
2 --     estudiante_id: 1=Miguel, 2=Sof a, 3=H ctor
3 --     curso_id:      1=Bases de Datos, 2= lgebra , 3=Redes
4
5 INSERT INTO inscripciones (estudiante_id, curso_id,
6     fecha_alta)
7 VALUES
8     (1, 1, '2024-08-20'), -- Miguel -> Bases de Datos
9     (1, 3, '2024-08-20'), -- Miguel -> Redes de C mputo
10    (2, 1, '2024-08-21'), -- Sof a -> Bases de Datos
11    (2, 2, '2024-08-21'), -- Sof a -> lgebra Lineal
12    (3, 1, '2024-08-22'), -- H ctor -> Bases de Datos
13    (3, 2, '2024-08-22'), -- H ctor -> lgebra Lineal
14    (3, 3, '2024-08-22'); -- H ctor -> Redes de
15                                C mputo
```

Listing 66: Paso 2 — insertar en la tabla intermedia

```
1 -- La calificacion es un atributo de la RELACI N, no de
2 --     ninguna entidad sola
3 UPDATE inscripciones
4 SET     calificacion = 9.2
5 WHERE   estudiante_id = 1
6 AND     curso_id      = 1;
```

Listing 67: Paso 3 — actualizar atributos propios de la relación

Ejemplo práctico

Verificar las inscripciones de un estudiante con una consulta de control:

```
1 SELECT e.nombre      AS estudiante,
2        c.titulo      AS curso,
3        i.fecha_alta,
4        i.calificacion
5 FROM   inscripciones AS i
6 INNER JOIN estudiantes AS e ON i.estudiante_id = e.
7        estudiante_id
8 INNER JOIN cursos      AS c ON i.curso_id      = c.
9        curso_id
10 WHERE e.nombre = 'Miguel R os';
```

Listing 68: Consulta de verificación post-inserción N:M

Nota importante

La **clave primaria compuesta** (estudiante_id, curso_id) en la tabla intermedia garantiza que un mismo par no pueda insertarse dos veces, cumpliendo

la regla de integridad de entidad sin necesidad de un campo `AUTO_INCREMENT` adicional.

Resumen del módulo

Tabla 11: Estrategias de inserción según la cardinalidad de la relación

Relación	Tablas involucradas	Consideración clave
1:1	Padre + hijo con FK UNIQUE	Insertar padre primero; usar <code>LAST_INSERT_ID()</code>
1:N	Padre + uno o más hijos	Todos los hijos referencian el mismo padre
N:M	Dos padres + tabla intermedia	Insertar ambos padres antes de poblar la tabla pivote

10. Escritura de Datos

Las sentencias de escritura forman parte del sublenguaje **DML** (*Data Manipulation Language*) de SQL. Permiten insertar, modificar y eliminar filas en las tablas de una base de datos relacional.

Nota importante

Toda operación DML es transaccional. En motores como MySQL/InnoDB o PostgreSQL, los cambios pueden deshacerse con **ROLLBACK** mientras la transacción no haya sido confirmada con **COMMIT**.

10.1. INSERT

Definición: INSERT

Agrega **una o más filas** nuevas a una tabla. Los valores proporcionados deben respetar los tipos de datos, restricciones **NOT NULL** y claves foráneas definidas en el esquema.

Sintaxis — una sola fila:

```
1 INSERT INTO nombre_tabla (col1, col2, col3)
2 VALUES (val1, val2, val3);
```

Listing 69: INSERT de una fila con columnas explícitas

```
1 -- Los valores deben seguir exactamente el orden de la
   tabla
2 INSERT INTO nombre_tabla
3 VALUES (val1, val2, val3);
```

Listing 70: INSERT sin lista de columnas (no recomendado)

Nota importante

Especificar siempre la lista de columnas hace el código más robusto ante futuros cambios en el esquema (columnas nuevas, reordenamiento, etc.).

Sintaxis — múltiples filas:

```
1 INSERT INTO nombre_tabla (col1, col2, col3)
2 VALUES
3     (val1a, val2a, val3a),
4     (val1b, val2b, val3b),
5     (val1c, val2c, val3c);
```

Listing 71: INSERT de varias filas en una sola sentencia

Ejemplo práctico

Poblar la tabla `productos` con tres artículos en una sola operación:

```
1 INSERT INTO productos (nombre, precio, stock,  
2 categoria_id)  
3 VALUES  
4 ('Teclado mecánico', 850.00, 40, 3),  
5 ('Monitor 24"', 3200.00, 15, 3),  
6 ('Silla ergonómica', 5400.00, 8, 5);
```

Listing 72: Inserción múltiple en la tabla `productos`

Los campos marcados con `AUTO_INCREMENT` (`producto_id`) se omiten; el motor los genera automáticamente.

Inserción a partir de un `SELECT`:

```
1 INSERT INTO productos_archivo (nombre, precio, categoria_id  
2 )  
3 SELECT nombre, precio, categoria_id  
4 FROM productos  
5 WHERE stock = 0;
```

Listing 73: `INSERT ... SELECT`: copiar filas desde otra tabla

10.2. UPDATE

Definición: `UPDATE`

Modifica los valores de **una o más columnas** en las filas que satisfacen la condición `WHERE`. Si se omite `WHERE`, la actualización afecta a **todas** las filas de la tabla.

Sintaxis general:

```
1 UPDATE nombre_tabla  
2 SET col1 = nuevo_val1,  
3 col2 = nuevo_val2  
4 WHERE condicion;
```

Listing 74: Sintaxis de `UPDATE` con `WHERE`

Ejemplo práctico

Aplicar un incremento de 10 % al precio de todos los productos de la categoría 3:

```
1 UPDATE productos  
2 SET precio = precio * 1.10  
3 WHERE categoria_id = 3;
```

Listing 75: `UPDATE` con expresión aritmética

Ejemplo práctico

Actualizar varios campos de un registro específico:

```
1 UPDATE productos
2 SET     nombre  = 'Teclado mecánico RGB',
3         stock   = 55,
4         precio  = 920.00
5 WHERE  producto_id = 1;
```

Listing 76: UPDATE de múltiples columnas por clave primaria

Nota importante

Buenas prácticas antes de ejecutar un UPDATE:

1. Verificar la condición con un SELECT previo usando el mismo WHERE.
2. Ejecutar dentro de una transacción explícita para poder hacer ROLLBACK si el resultado no es el esperado.
3. En MySQL, activar `sql_safe_updates = 1` para bloquear UPDATE sin WHERE o sin clave en la condición.

UPDATE con JOIN (MySQL / MariaDB):

```
1 UPDATE productos AS p
2 INNER JOIN categorias AS c
3     ON p.categoria_id = c.categoria_id
4 SET p.precio = p.precio * 0.90
5 WHERE c.nombre = 'Electrónica';
```

Listing 77: UPDATE que referencia otra tabla mediante JOIN

10.3. DELETE

Definición: DELETE

Elimina las filas que cumplen la condición WHERE. Al igual que UPDATE, si se omite WHERE se eliminan **todas** las filas de la tabla, aunque la estructura y los metadatos se conservan.

Sintaxis general:

```
1 DELETE FROM nombre_tabla
2 WHERE condicion;
```

Listing 78: Sintaxis de DELETE con WHERE

Ejemplo práctico

Eliminar los productos sin stock que pertenecen a la categoría 5:

```
1 DELETE FROM productos
```

```

2 WHERE stock = 0
3 AND categoria_id = 5;

```

Listing 79: DELETE con condición compuesta

DELETE con JOIN (MySQL / MariaDB):

```

1 DELETE p
2 FROM productos AS p
3 INNER JOIN categorias AS c
4 ON p.categoria_id = c.categoria_id
5 WHERE c.activa = 0;

```

Listing 80: DELETE que filtra mediante una tabla relacionada

Diferencias entre DELETE y TRUNCATE

Tabla 12: Comparación entre DELETE y TRUNCATE

Característica	DELETE	TRUNCATE
Sublenguaje SQL	DML	DDL
Cláusula WHERE	Sí — elimina filas selectivamente	No — elimina todas las filas
Transaccional	Sí — admite ROLLBACK	Depende del motor; en MySQL no
Velocidad	Más lenta (registro fila a fila)	Más rápida (libera páginas de datos)
Reinicio de AUTO_INCREMENT	No — el contador no se reinicia	Sí — el contador vuelve a 1
Activación de <i>triggers</i>	Sí	No en la mayoría de motores
Restricciones de FK	Respeto ON DELETE definido en el FK	Falla si existen referencias activas

```

1 -- Elimina solo las filas del a o 2020 (reversible)
2 DELETE FROM ventas
3 WHERE YEAR(fecha) = 2020;
4
5 -- Vac a la tabla completa y reinicia el auto_increment (
  irreversible en MySQL)
6 TRUNCATE TABLE ventas;

```

Listing 81: Diferencia práctica entre DELETE y TRUNCATE

Nota importante

Ante la duda entre **DELETE** y **TRUNCATE**, preferir **DELETE** dentro de una transacción explícita: ofrece mayor control y trazabilidad. Reservar **TRUNCATE** para entornos de desarrollo o procesos de carga masiva donde la velocidad es crítica y no se requiere reversibilidad.

Resumen del módulo

Tabla 13: Sentencias DML de escritura

Sentencia	Acción	Consideración clave
INSERT	Agregar filas	Especificar siempre la lista de columnas
INSERT ...SELECT	Copiar filas	Útil para archivado o migración de datos
UPDATE	Modificar filas	Nunca omitir WHERE ; verificar con SELECT previo
DELETE	Eliminar filas	Transaccional; respeta triggers y claves foráneas
TRUNCATE	Vaciar tabla completa	DDL, más rápido, pero no reversible en MySQL

11. Administración de la Base de Datos

Las sentencias de administración de bases de datos pertenecen al sublenguaje **DDL**. Operan sobre el contenedor de más alto nivel en MySQL: la **base de datos** (también llamada *schema*), que agrupa tablas, vistas, procedimientos y demás objetos.

11.1. CREATE DATABASE

Definición: CREATE DATABASE

Crea una nueva base de datos vacía en el servidor. Permite especificar el **juego de caracteres** (*character set*) y la **colación** (*collation*) que determinarán cómo se almacenan y comparan las cadenas de texto en todas sus tablas.

```
1 CREATE DATABASE [IF NOT EXISTS] nombre_bd
2   [CHARACTER SET charset_name]
3   [COLLATE collation_name];
```

Listing 82: Sintaxis de CREATE DATABASE

Ejemplo práctico

Crear una base de datos para una aplicación en español con soporte completo de Unicode:

```
1 CREATE DATABASE IF NOT EXISTS tienda_online
2   CHARACTER SET utf8mb4
3   COLLATE utf8mb4_unicode_ci;
4
5 -- Seleccionar la base de datos activa
6 USE tienda_online;
```

Listing 83: Base de datos con utf8mb4 y collation en español

Juegos de caracteres y colaciones más comunes:

Tabla 14: Character sets y collations habituales en MySQL

charset	collation	Uso recomendado
utf8mb4	utf8mb4_unicode_ci	Unicode completo; soporta emojis; recomendado
utf8mb4	utf8mb4_0900_ai_ci	Unicode 9.0; sin acento/mayúsculas; MySQL 8.0+
utf8mb4	utf8mb4_bin	Comparación binaria; distingue mayúsculas
latin1	latin1_swedish_ci	Legado; solo caracteres ISO-8859-1

Nota importante

Usar `utf8` (sin `mb4`) en MySQL es un error frecuente: ese alias solo soporta hasta 3 bytes por carácter y excluye emojis y algunos ideogramas CJK. La elección correcta para proyectos nuevos es siempre `utf8mb4`.

```
1 SHOW DATABASES;                -- listar todas las
    bases de datos
2 SHOW CREATE DATABASE tienda_online; -- ver la definici n
    completa
3 SELECT DATABASE();              -- ver la base de datos
    activa
```

Listing 84: Comandos de exploración de bases de datos

11.2. DROP DATABASE

Definición: DROP DATABASE

Elimina la base de datos y **todos los objetos que contiene** (tablas, vistas, procedimientos, datos) de forma permanente e irreversible. No existe confirmación ni papelera de reciclaje.

```
1 DROP DATABASE [IF EXISTS] nombre_bd;
```

Listing 85: Sintaxis de DROP DATABASE

Ejemplo práctico

```
1 -- Genera error si la base de datos no existe
2 DROP DATABASE tienda_online;
3
4 -- No genera error si no existe ( til en scripts de
    despliegue)
5 DROP DATABASE IF EXISTS tienda_online;
```

Listing 86: DROP DATABASE con y sin guarda

Nota importante

Precauciones antes de ejecutar DROP DATABASE:

1. Verificar que se tiene un respaldo (*backup*) actualizado.
2. Confirmar que ninguna aplicación activa tiene conexiones abiertas a esa base de datos.
3. Comprobar el nombre con `SHOW DATABASES` antes de ejecutar.
4. En producción, revocar el privilegio `DROP` a usuarios de aplicación para evitar ejecuciones accidentales.

12. Conceptos Avanzados

Los objetos descritos en este módulo permiten construir bases de datos más eficientes, seguras y mantenibles. Aunque no son imprescindibles para consultas básicas, son herramientas fundamentales en entornos de producción.

12.1. INDEX

Definición: Índice

Estructura de datos auxiliar que el motor mantiene sincronizada con la tabla para acelerar las operaciones de búsqueda, ordenamiento y unión. Mejora el rendimiento de **SELECT** a cambio de un coste adicional en **INSERT**, **UPDATE** y **DELETE**, y de espacio en disco.

Tipos de índice en MySQL:

Tabla 15: Tipos de índice disponibles en MySQL/InnoDB

Tipo	Motor	Uso recomendado
B-TREE	InnoDB, MyISAM	Búsquedas exactas, rangos, ORDER BY
HASH	Memory	Solo igualdad exacta; no soporta rangos
FULLTEXT	InnoDB (5.6+)	Búsqueda de texto libre en columnas TEXT
SPATIAL	InnoDB, MyISAM	Datos geoespaciales (GEOMETRY)

```

1  -- índice simple en una columna
2  CREATE INDEX idx_apellido ON clientes(apellido);
3
4  -- índice compuesto (el orden de columnas importa)
5  CREATE INDEX idx_cat_precio ON productos(categoria_id,
6      precio);
7
8  -- índice UNIQUE (equivale a restricción UNIQUE)
9  CREATE UNIQUE INDEX uq_correo ON usuarios(correo);
10
11 -- índice FULLTEXT para búsqueda de texto
12 CREATE FULLTEXT INDEX ft_descripcion ON productos(
13     descripcion);
14
15 -- Eliminar un índice
16 DROP INDEX idx_apellido ON clientes;
```

Listing 87: Creación y eliminación de índices

```

1  SELECT nombre, descripcion
2  FROM    productos
3  WHERE   MATCH(descripcion) AGAINST('teclado mecánico' IN
4      BOOLEAN MODE);
```

Listing 88: Uso del índice FULLTEXT con MATCH ... AGAINST

Nota importante

Cuándo crear un índice:

- Columnas que aparecen frecuentemente en WHERE, JOIN ON, ORDER BY o GROUP BY.
- Columnas con alta cardinalidad (muchos valores distintos).

Cuándo evitarlo:

- Tablas pequeñas (el escaneo completo es más rápido).
- Columnas con muy baja cardinalidad (activo, genero).
- Tablas con escrituras masivas y pocas lecturas.

12.2. TRIGGER

Definición: TRIGGER

Bloque de código SQL que se ejecuta **automáticamente** en respuesta a un evento DML (INSERT, UPDATE o DELETE) sobre una tabla, antes (BEFORE) o después (AFTER) de que el cambio se aplique.

```

1 CREATE TRIGGER nombre_trigger
2   { BEFORE | AFTER } { INSERT | UPDATE | DELETE }
3   ON nombre_tabla
4   FOR EACH ROW
5 BEGIN
6   -- cuerpo del trigger
7   -- NEW.columna -> valor nuevo (INSERT / UPDATE)
8   -- OLD.columna -> valor anterior (UPDATE / DELETE)
9 END;
```

Listing 89: Sintaxis general de un TRIGGER

Ejemplo práctico

Registrar en una tabla de auditoría cada cambio de precio en productos:

```

1 CREATE TABLE auditoria_precios (
2   id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY
3   ,
4   producto_id INT UNSIGNED NOT NULL ,
5   precio_ant  DECIMAL(10,2) ,
6   precio_new  DECIMAL(10,2) ,
7   cambiado_en DATETIME NOT NULL DEFAULT
      CURRENT_TIMESTAMP
8 );
```

```

8
9 DELIMITER $$
10 CREATE TRIGGER trg_auditoria_precio
11     AFTER UPDATE ON productos
12     FOR EACH ROW
13 BEGIN
14     IF OLD.precio <> NEW.precio THEN
15         INSERT INTO auditoria_precios
16             (producto_id, precio_ant, precio_new)
17         VALUES (NEW.producto_id, OLD.precio, NEW.precio
18             );
19     END IF;
20 END$$
21 DELIMITER ;

```

Listing 90: TRIGGER de auditoría de cambios de precio

Nota importante

Los triggers dificultan la depuración y pueden crear efectos secundarios inesperados si no se documentan. Úsarlos con moderación; para lógica compleja, preferir procedimientos almacenados invocados explícitamente.

12.3. VIEW**Definición: Vista (*View*)**

Consulta **SELECT** almacenada con nombre propio que puede usarse como si fuera una tabla. No duplica datos: cada vez que se consulta, el motor ejecuta la consulta subyacente. Simplifica consultas complejas, encapsula lógica y puede restringir el acceso a columnas sensibles.

```

1  -- Crear la vista
2  CREATE VIEW vista_pedidos_activos AS
3      SELECT p.pedido_id,
4             c.nombre      AS cliente,
5             p.total,
6             p.fecha
7      FROM   pedidos      AS p
8      INNER JOIN clientes AS c ON p.cliente_id = c.cliente_id
9      WHERE  p.estado = 'activo';
10
11  -- Usar la vista exactamente como una tabla
12  SELECT cliente, SUM(total) AS total_compras
13  FROM    vista_pedidos_activos
14  GROUP BY cliente
15  ORDER BY total_compras DESC;
16
17  -- Reemplazar o eliminar la vista

```

```

18 CREATE OR REPLACE VIEW vista_pedidos_activos AS ...;
19 DROP VIEW IF EXISTS vista_pedidos_activos;

```

Listing 91: Creación y consulta de una vista

Nota importante

Las vistas en MySQL no almacenan datos en caché por defecto. Para escenarios de alto rendimiento con consultas agregadas costosas, considerar **vistas materializadas** (disponibles en PostgreSQL) o tablas de resumen actualizadas por eventos programados.

12.4. STORED PROCEDURE

Definición: Procedimiento almacenado

Bloque de código SQL con nombre, almacenado en el servidor y ejecutable mediante CALL. Acepta parámetros de entrada (IN), salida (OUT) o ambos (INOUT), y permite estructuras de control (IF, WHILE, LOOP, cursores).

```

1 DELIMITER $$
2 CREATE PROCEDURE sp_resumen_cliente(
3     IN p_cliente_id INT UNSIGNED,
4     OUT p_total_pedidos INT,
5     OUT p_monto_total DECIMAL(12,2)
6 )
7 BEGIN
8     SELECT COUNT(*),
9            COALESCE(SUM(total), 0)
10    INTO    p_total_pedidos,
11           p_monto_total
12    FROM    pedidos
13    WHERE   cliente_id = p_cliente_id;
14 END$$
15 DELIMITER ;
16
17 -- Invocar el procedimiento
18 CALL sp_resumen_cliente(7, @num_pedidos, @monto);
19 SELECT @num_pedidos AS pedidos, @monto AS monto_total;

```

Listing 92: Procedimiento con parámetros IN y OUT

Ejemplo práctico

Procedimiento con flujo de control para aplicar descuentos escalonados:

```

1 DELIMITER $$
2 CREATE PROCEDURE sp_aplicar_descuento(
3     IN p_producto_id INT UNSIGNED,
4     IN p_porcentaje DECIMAL(5,2)
5 )

```

```

6 BEGIN
7     IF p_porcentaje <= 0 OR p_porcentaje > 80 THEN
8         SIGNAL SQLSTATE '45000'
9         SET MESSAGE_TEXT = 'Porcentaje fuera de
            rango permitido';
10    ELSEIF p_porcentaje <= 20 THEN
11        UPDATE productos
12        SET     precio = precio * (1 - p_porcentaje /
13            100)
14        WHERE  producto_id = p_producto_id;
15    ELSE
16        -- Descuento mayor requiere aprobaci n: solo
17        registrar solicitud
18        INSERT INTO solicitudes_descuento(producto_id,
19            porcentaje)
20        VALUES (p_producto_id, p_porcentaje);
21    END IF;
22 END$$
23 DELIMITER ;

```

Listing 93: Stored procedure con IF / ELSEIF

12.5. Transacciones

Definición: Transacción

Unidad lógica de trabajo que agrupa una o más operaciones DML. Garantiza las propiedades **ACID**: Atomicidad, Consistencia, Aislamiento y Durabilidad. Si alguna operación falla, todo el grupo puede deshacerse.

Tabla 16: Comandos de control de transacciones

Comando	Efecto
START TRANSACTION / BEGIN	Inicia una transacción explícita
COMMIT	Confirma y persiste todos los cambios
ROLLBACK	Deshace todos los cambios desde el último COMMIT
SAVEPOINT nombre	Crea un punto de restauración parcial dentro de la transacción
ROLLBACK TO nombre	Deshace hasta el savepoint indicado (sin terminar la transacción)
RELEASE SAVEPOINT nombre	Elimina un savepoint sin hacer rollback

```
1 START TRANSACTION;
```

```
2      INSERT INTO pedidos (cliente_id, total, fecha)
3      VALUES (12, 1500.00, CURDATE());
4
5
6      SAVEPOINT sp_pedido_insertado;
7
8      INSERT INTO detalle_pedido (pedido_id, producto_id,
9      cantidad)
10     VALUES (LAST_INSERT_ID(), 5, 3);
11
12     -- Si este UPDATE falla, se puede revertir solo el
13     -- detalle
14     UPDATE productos SET stock = stock - 3 WHERE
15     producto_id = 5;
16
17     -- Todo correcto: confirmar
18     COMMIT;
19
20     -- En caso de error detectado en la aplicaci n:
21     -- ROLLBACK TO sp_pedido_insertado; -- deshace solo desde
22     -- el savepoint
23     -- ROLLBACK; -- deshace toda la
24     -- transacci n
```

Listing 94: Transacción con SAVEPOINT y manejo de errores

Nota importante

MySQL opera en modo `AUTOCOMMIT = 1` por defecto: cada sentencia DML es una transacción implícita que se confirma de inmediato. `START TRANSACTION` suspende ese modo hasta que se ejecute `COMMIT` o `ROLLBACK`.

12.6. Concurrency

Definición: Concurrency

Capacidad del motor para atender **múltiples transacciones simultáneas** sin que interfieran entre sí. Se gestiona mediante **bloqueos** (*locks*) y **niveles de aislamiento** (*isolation levels*).

Niveles de aislamiento en SQL estándar:

Tabla 17: Niveles de aislamiento y fenómenos que previenen

Nivel	Descri resu- mida	<i>Dirty read</i>	<i>Non-rep. read</i>	<i>Phantom</i>
READ UNCOMMITTED	Ve cam- bios no con- firma- dos de otras tran- sac- cio- nes	Sí	Sí	Sí
READ COMMITTED	Solo ve cam- bios ya con- firma- dos	No	Sí	Sí
REPEATABLE READ	Garant: lec- turas con- sis- ten- tes (de- fecto In- noDB)	No	No	Sí
SERIALIZABLE	Ejecuci se- cuen- cial total; má- ximo aisla- mien- to	No	No	No

```
1  -- Ver el nivel actual
2  SELECT @@transaction_isolation;
3
4  -- Cambiar para la sesi n actual
5  SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
6
7  -- Cambiar globalmente (requiere privilegio SUPER)
8  SET GLOBAL TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Listing 95: Configurar y consultar el nivel de aislamiento

Nota importante

Deadlock: dos transacciones se bloquean mutuamente esperando recursos que la otra retiene. InnoDB detecta el ciclo automáticamente y aborta la transacción de menor coste. La aplicación debe estar preparada para reintentar la operación al recibir el error `ER_LOCK_DEADLOCK` (1213). Para minimizarlos: acceder siempre a las tablas en el mismo orden y mantener las transacciones lo más cortas posible.

13. Conexión desde Código

Integrar una base de datos en una aplicación requiere un **conector** o *driver*: biblioteca que traduce las llamadas del lenguaje de programación al protocolo de red del motor SQL.

13.1. Conectores y Drivers

Tabla 18: Conectores oficiales y populares por lenguaje

Lenguaje	Librería / Driver	Instalación / Referencia
Python	mysql-connector-python	pip install mysql-connector-python
Python	PyMySQL	pip install pymysql
Python	SQLAlchemy (ORM)	pip install sqlalchemy
Node.js	mysql2	npm install mysql2
Node.js	Prisma (ORM)	npm install prisma
Java	MySQL Connector/J	Dependencia Maven: mysql:mysql-connector-java
PHP	PDO_MySQL	Incluido en PHP; activar extensión pdo_mysql
Go	go-sql-driver/mysql	go get github.com/go-sql-driver/mysql

Ejemplo práctico

Conexión y consulta básica con Python (mysql-connector-python):

```

1  import mysql.connector
2
3  conn = mysql.connector.connect(
4      host      = "localhost",
5      user      = "app_user",
6      password  = "s3cr3to",
7      database  = "tienda_online"
8  )
9
10 cursor = conn.cursor(dictionary=True)
11 cursor.execute("SELECT producto_id, nombre, precio FROM
12                 productos LIMIT 5")
13
14 for fila in cursor.fetchall():
15     print(fila["nombre"], fila["precio"])
16
17 cursor.close()
18 conn.close()

```

Listing 96: Conexión a MySQL desde Python

Ejemplo práctico

Conexión con Node.js usando mysql2 y async/await:

```
1  const mysql = require('mysql2/promise');
2
3  const conn = await mysql.createConnection({
4    host:      'localhost',
5    user:      'app_user',
6    password:  's3cr3to',
7    database:  'tienda_online'
8  });
9
10 const [rows] = await conn.execute(
11   'SELECT nombre, precio FROM productos WHERE
12     categoria_id = ?',
13   [3] // par metro enlazado (previene SQL
14       Injection)
15 );
16 console.table(rows);
17 await conn.end();
```

Listing 97: Conexión a MySQL desde Node.js

13.2. SQL Injection

Definición: SQL Injection

Vulnerabilidad que permite a un atacante **inyectar código SQL arbitrario** en una consulta, manipulando su lógica para extraer, modificar o destruir datos, o para eludir la autenticación.

Ejemplo práctico

Código vulnerable: concatenación directa de entrada del usuario.

```
1  usuario = input("Usuario: ") # atacante ingresa: '
   OR '1'='1
2  query = "SELECT * FROM usuarios WHERE username = '" +
   usuario + "'"
3  # Consulta resultante:
4  # SELECT * FROM usuarios WHERE username = '' OR '1'='1'
5  # -> devuelve TODOS los usuarios sin contrase a
6  cursor.execute(query)
```

Listing 98: Código VULNERABLE a SQL Injection

Ejemplo práctico

Código seguro: consultas parametrizadas (*prepared statements*).

```
1 usuario = input("Usuario: ")
2 # El driver escapa automáticamente el valor
3 cursor.execute(
4     "SELECT * FROM usuarios WHERE username = %s",
5     (usuario,) # par metro enlazado, nunca
6               concatenado
7 )
```

Listing 99: Prevención con consultas parametrizadas

Medidas de prevención complementarias:

- Usar **siempre** consultas parametrizadas o ORMs que las generen internamente.
- Aplicar el **principio de mínimo privilegio**: el usuario de la aplicación solo debe tener los permisos estrictamente necesarios (SELECT, INSERT, etc.), nunca DROP o GRANT.
- Validar y sanitizar la entrada en la capa de aplicación antes de pasarla al driver.
- Evitar exponer mensajes de error detallados del motor en respuestas de la API.

14. Otros Clientes SQL

Existen múltiples herramientas gráficas y de línea de comandos para interactuar con bases de datos SQL sin necesidad de escribir directamente en la terminal.

Tabla 19: Clientes SQL populares

Cliente	Plataforma	Características destacadas
DBeaver	Windows, macOS, Linux	Gratuito y de código abierto; soporta más de 80 motores
TablePlus	macOS, Windows, Linux, iOS	Interfaz nativa moderna; licencia de pago con prueba gratuita
DataGrip	Windows, macOS, Linux	IDE de JetBrains; refactorización SQL, inspecciones inteligentes
pgAdmin	Web / escritorio	Cliente oficial de PostgreSQL; gestión completa del servidor
Adminer	Web (un solo archivo PHP)	Ligero, sin instalación; útil en servidores remotos
MySQL Workbench	Windows, macOS, Linux	Cliente oficial de MySQL; diseño ER visual incluido

15. PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacionales **de código abierto**, con más de 35 años de desarrollo activo, reconocido por su robustez, extensibilidad y cumplimiento estricto del estándar SQL.

Diferencias clave con MySQL:

Tabla 20: MySQL vs. PostgreSQL — diferencias principales

Aspecto	MySQL	PostgreSQL
Licencia	GPL / comercial (Oracle)	PostgreSQL License (MIT-like)
Modelo MVCC	Solo en InnoDB	Nativo en todo el motor
Tipos de datos avanzados	Limitados	JSON/JSONB, Arrays, HSTORE, UUID nativo
Vistas materializadas	No nativo	Sí, con <code>REFRESH MATERIALIZED VIEW</code>
Full-text search	Básico	Avanzado; soporta diccionarios y ranking
Extensiones	Plugins limitados	PostGIS, pgcrypto, TimescaleDB, etc.
Procedimientos almacenados	SQL / MySQL dialect	PL/pgSQL, PL/Python, PL/Perl, etc.
Secuencias	<code>AUTO_INCREMENT</code>	<code>SERIAL</code> / <code>GENERATED ALWAYS AS IDENTITY</code>

Nota importante

PostgreSQL es generalmente preferido para aplicaciones que requieren consultas complejas, integridad de datos estricta o tipos de datos avanzados. MySQL sigue siendo muy popular en aplicaciones web de alto tráfico gracias a su velocidad en lecturas simples y su amplio soporte en servicios de alojamiento.

16. Despliegue en la Nube

Los servicios gestionados (*Database as a Service*, DBaaS) eliminan la carga operativa de instalar, parchear y respaldar el motor de base de datos.

Tabla 21: Servicios de base de datos en la nube

Servicio	Motor(es)	Características des- ta- ca- das
AWS RDS	MySQL, PostgreSQL, MariaDB, Oracle, SQL Server	Backups au- to- má- ti- cos, Multi- AZ, ré- pli- cas de lec- tura
AWS Aurora	MySQL / PostgreSQL compatible	Rendimiento has- ta 5× so- bre RDS; ser- ver- less dis- po- ni- ble
Google Cloud SQL	MySQL, PostgreSQL, SQL Server	Integración con Big- Query y Ver- tex AI
Azure Database	MySQL, PostgreSQL	Alta dis- po- ni- bili- dad con zona de

17. Próximos Pasos

Caminos de especialización

- **DBA (*Database Administrator*):** administración de servidores, alta disponibilidad, replicación, *tuning* de rendimiento, backups y recuperación ante desastres.
- **Data Engineer:** pipelines de datos (ETL/ELT), almacenes de datos (*data warehouses*), herramientas como dbt, Apache Spark, Airflow y servicios como BigQuery o Redshift.
- **Backend Developer:** integración de bases de datos en APIs REST y GraphQL, ORM avanzados, patrones de diseño de esquemas, caché con Redis.
- **Data Analyst / BI:** consultas analíticas avanzadas, funciones de ventana, herramientas de visualización (Metabase, Tableau, Power BI).

Recursos recomendados

- **Documentación oficial:** dev.mysql.com/doc y postgresql.org/docs.
- **Práctica interactiva:** [SQLZoo](#), [HackerRank SQL](#) y [LeetCode Database](#).
- **Libros de referencia:** *Learning MySQL* (Seyed M.M. & Jay Pipes), *PostgreSQL: Up and Running* (Regina O. Obe & Leo S. Hsu), *Designing Data-Intensive Applications* (Martin Kleppmann).
- **Modelado de datos:** dbdiagram.io para diseño visual de esquemas con notación DBML.